

TÉCNICAS DE APRENDIZADO DE MÁQUINA

Bacharelado em Sistemas da Informação

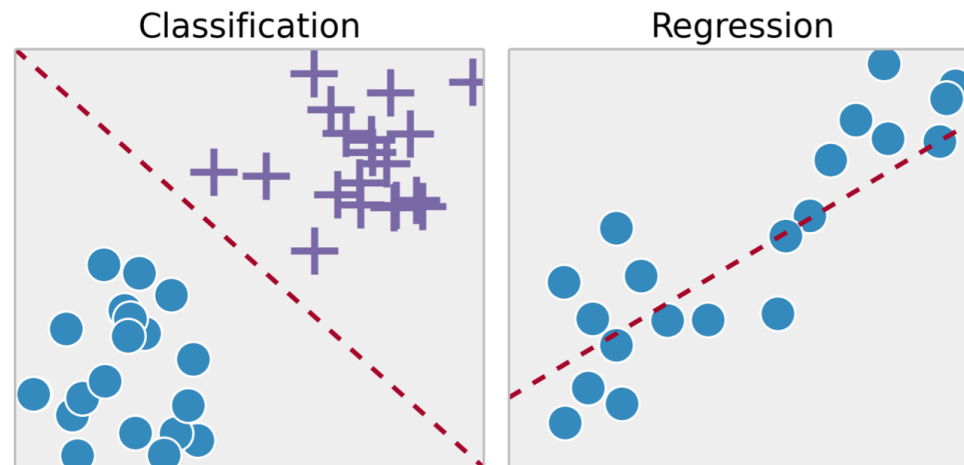
Prof. Marco André Abud Kappel

Aula 10 – Regressão Linear e Polinomial

Regressão Linear e Polinomial

- **Problemas de Regressão**

- A tarefa de regressão envolve a identificação de **padrões** em dados de forma a estabelecer qual é o melhor **modelo** que descreve seus comportamentos.
- Com este modelo, será possível fazer previsões de **valores numéricos** para uma variável, usando uma combinação das outras.

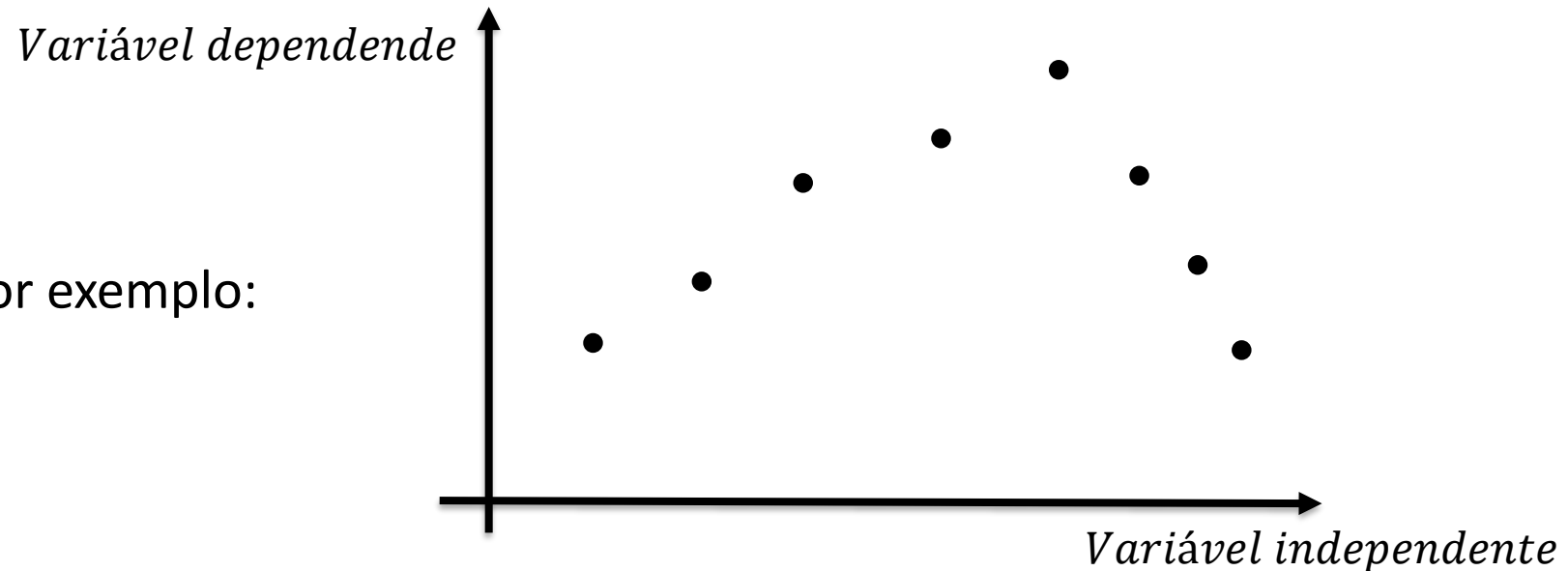


Regressão Linear e Polinomial

- **Problemas de Regressão**

- Assim como nos problemas de classificação, teremos um conjunto de **variáveis previsoras**.
- A principal diferença é que a **variável objetivo** não é mais uma classe, mas uma variação numérica.

➤ Por exemplo:

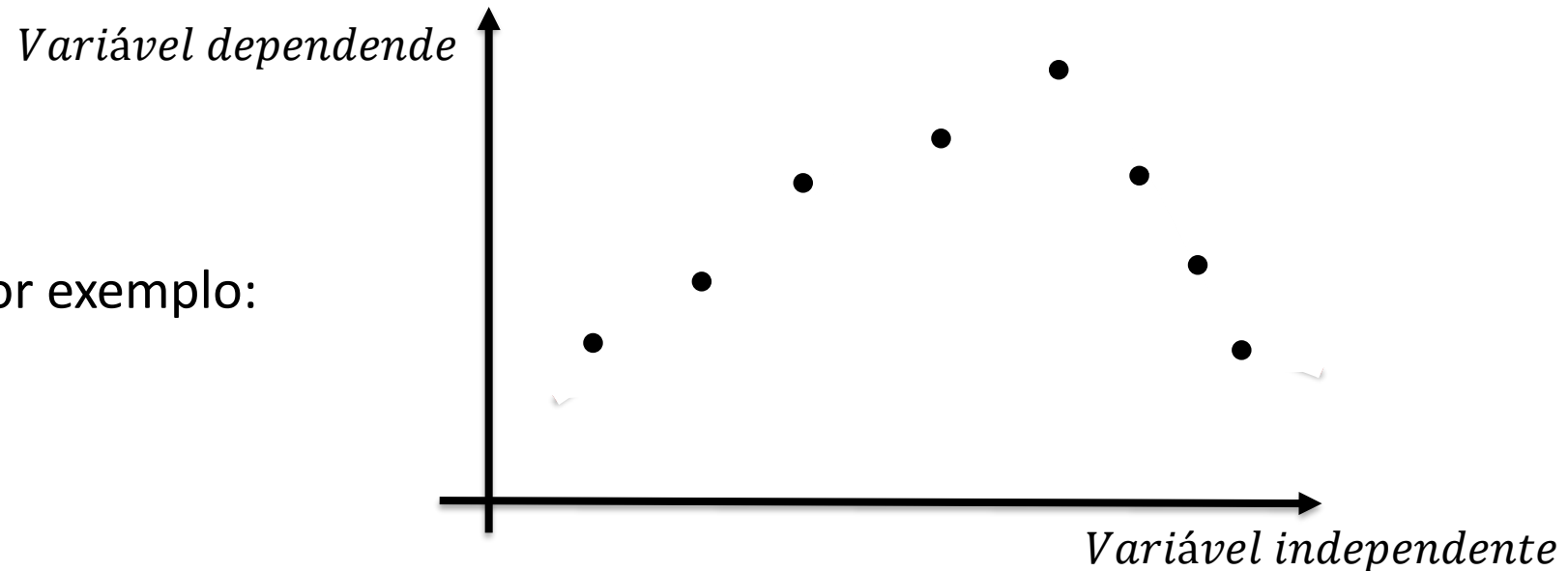


Regressão Linear e Polinomial

- **Problemas de Regressão**

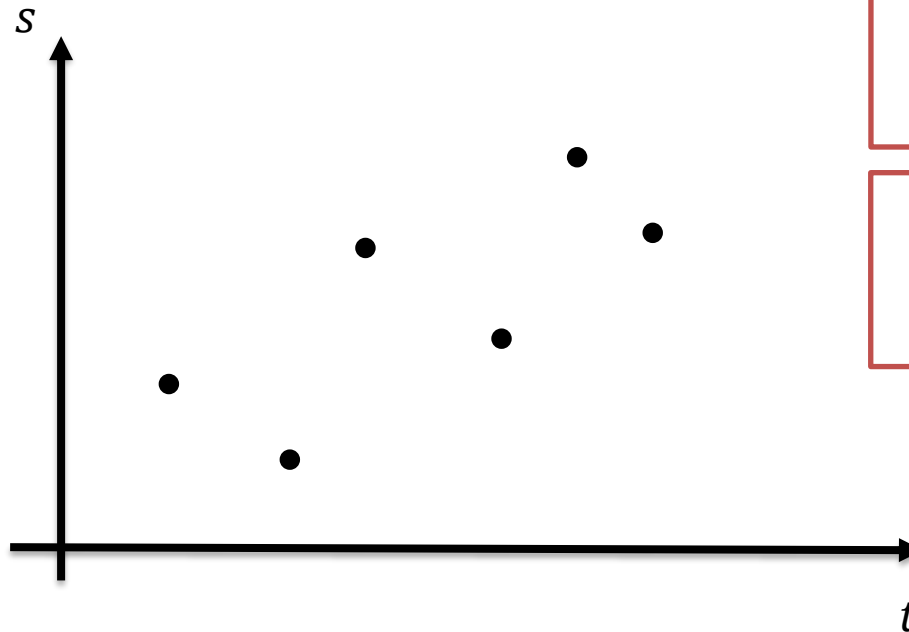
- Nós queremos descobrir qual é **modelo** que descreve melhor a tendência da **variável dependente**, usando todas as **variáveis independentes**.

➤ Por exemplo:



Regressão Linear e Polinomial

- Regressão linear
 - Exemplo simples: movimento linear



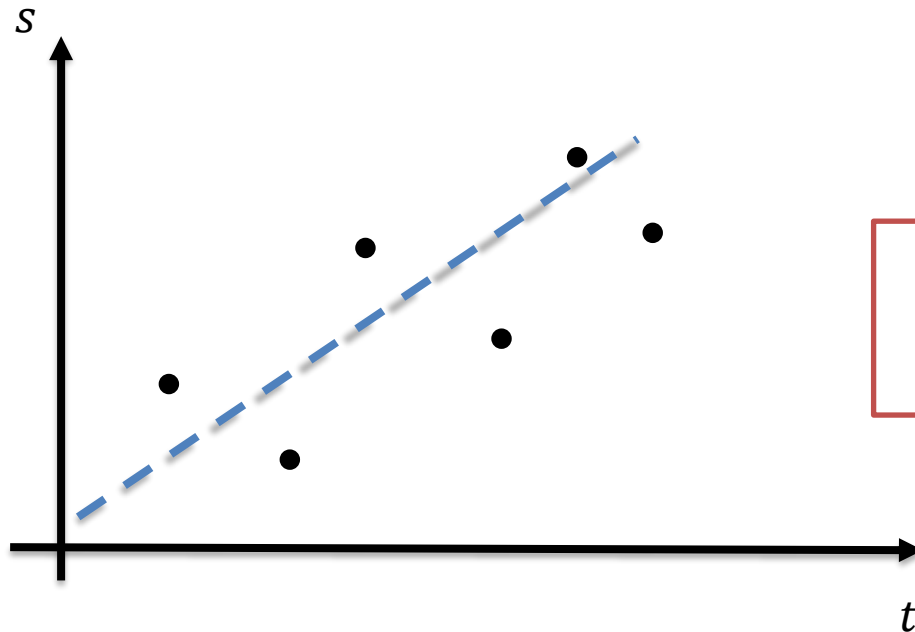
Vamos supor que temos os dados experimentais referentes ao **movimento de um objeto**.

Queremos descobrir um **padrão** nos dados que possibilite a **previsão** do comportamento.

Regressão Linear e Polinomial

- **Regressão linear**

- Com a **Regressão Linear**, iremos supor que o movimento pode ser descrito por uma reta.



$$y = ax + b$$

Porém, uma reta pode assumir **várias formas** de acordo com seus **parâmetros** ou **coeficientes**.

Regressão Linear e Polinomial

- **Regressão linear**
 - **Exemplo simples:** movimento linear

$$s(t) = 3t + 2$$

Quando os parâmetros da reta são definidos, é possível prever o comportamento do objeto em todo o trajeto.



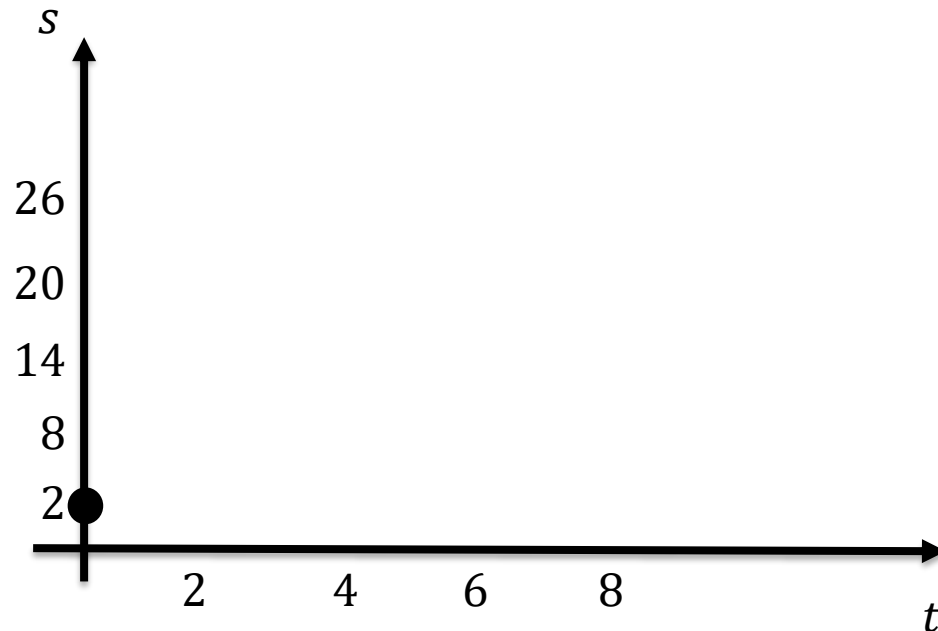
t	s

Regressão Linear e Polinomial

- **Regressão linear**
 - **Exemplo simples:** movimento linear

$$s(t) = 3t + 2$$

Quando os parâmetros da reta são definidos, é possível prever o comportamento do objeto em todo o trajeto.



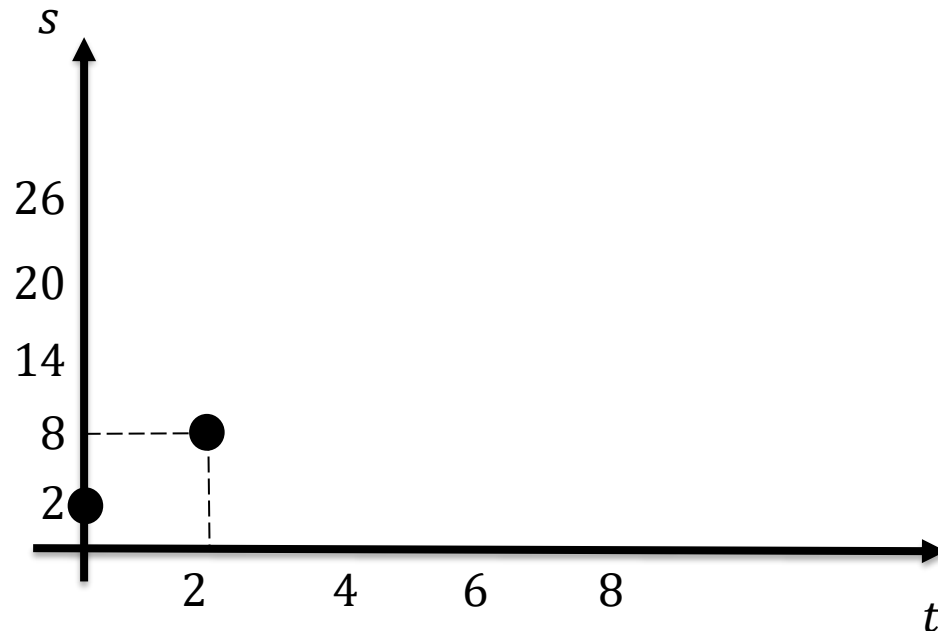
t	s
0	2

Regressão Linear e Polinomial

- **Regressão linear**
 - **Exemplo simples:** movimento linear

$$s(t) = 3t + 2$$

Quando os parâmetros da reta são definidos, é possível prever o comportamento do objeto em todo o trajeto.



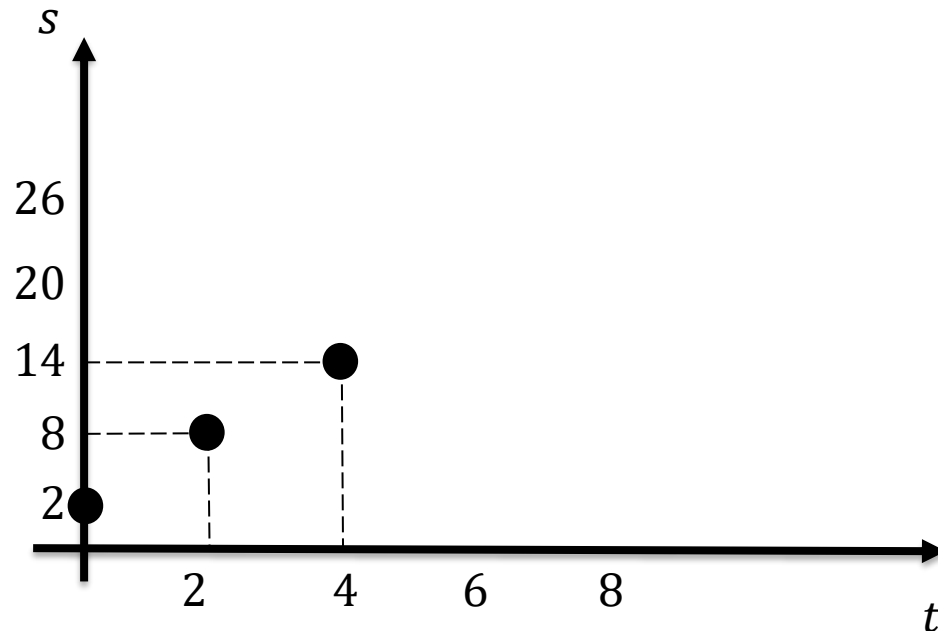
t	s
0	2
2	8

Regressão Linear e Polinomial

- **Regressão linear**
 - **Exemplo simples:** movimento linear

$$s(t) = 3t + 2$$

Quando os parâmetros da reta são definidos, é possível prever o comportamento do objeto em todo o trajeto.



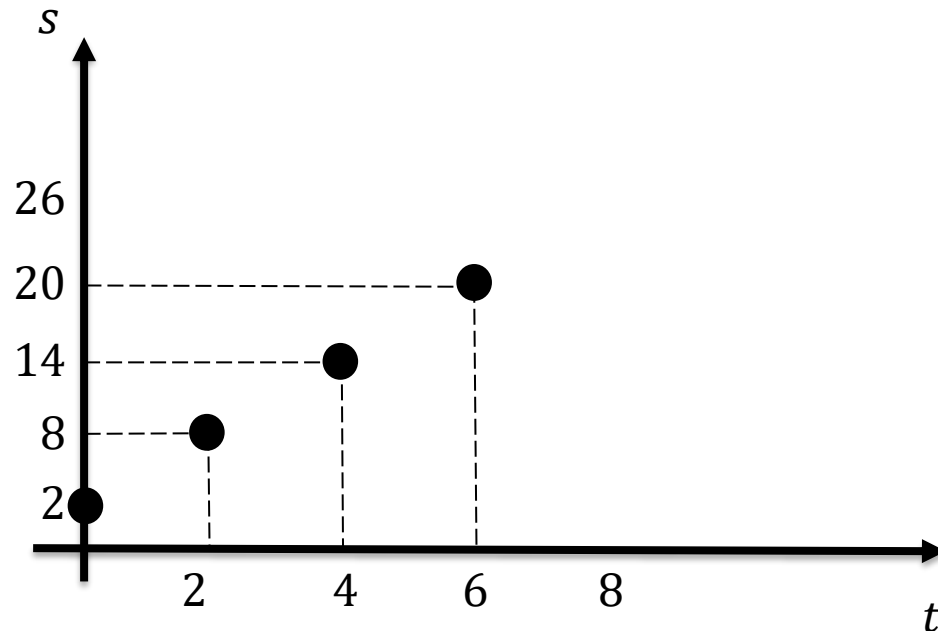
t	s
0	2
2	8
4	14

Regressão Linear e Polinomial

- **Regressão linear**
 - **Exemplo simples:** movimento linear

$$s(t) = 3t + 2$$

Quando os parâmetros da reta são definidos, é possível prever o comportamento do objeto em todo o trajeto.



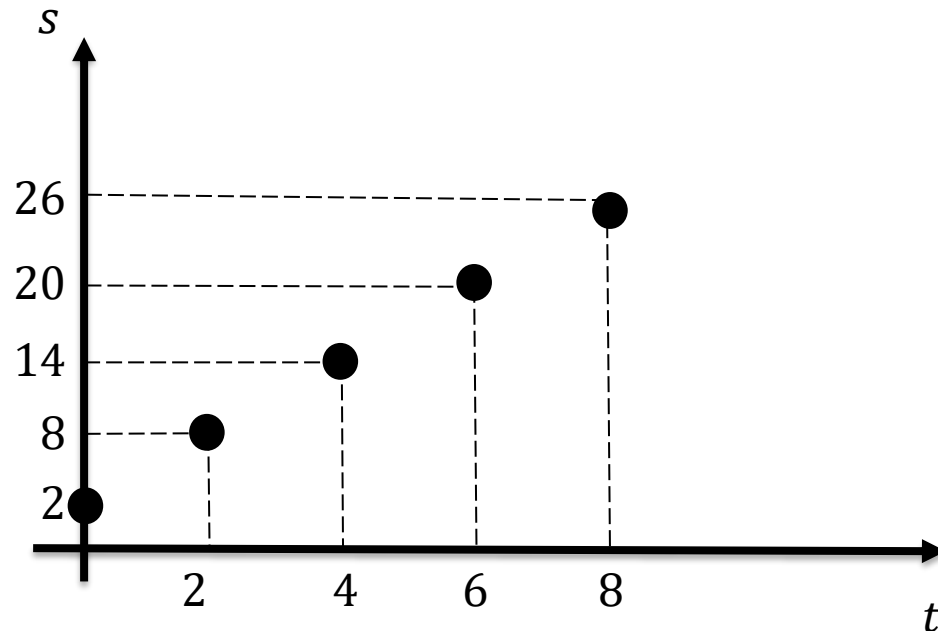
t	s
0	2
2	8
4	14
6	20

Regressão Linear e Polinomial

- **Regressão linear**
 - **Exemplo simples:** movimento linear

$$s(t) = 3t + 2$$

Quando os parâmetros da reta são definidos, é possível prever o comportamento do objeto em todo o trajeto.



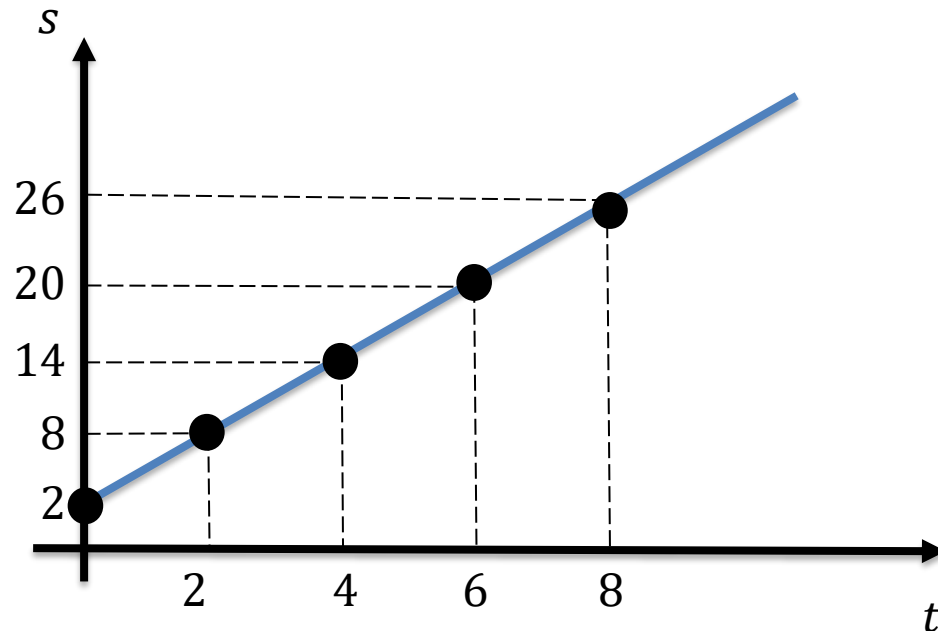
t	s
0	2
2	8
4	14
6	20
8	26

Regressão Linear e Polinomial

- **Regressão linear**
 - **Exemplo simples:** movimento linear

$$s(t) = 3t + 2$$

Quando os parâmetros da reta são definidos, é possível prever o comportamento do objeto em todo o trajeto.



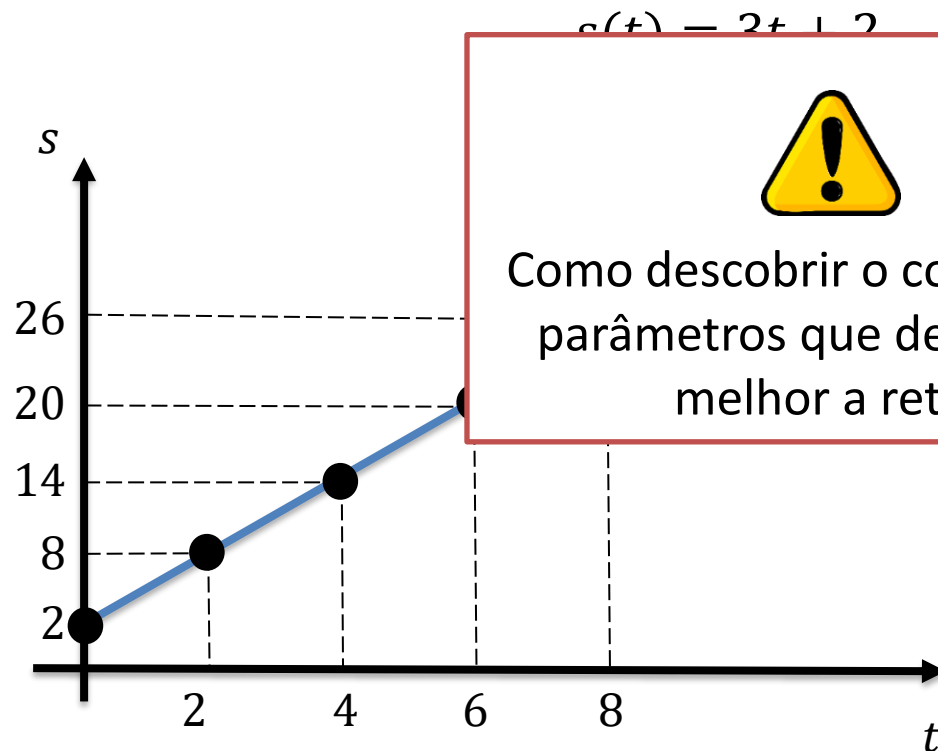
t	s
0	2
2	8
4	14
6	20
8	26

Regressão Linear e Polinomial

- **Regressão linear**

- **Exemplo simples:** movimento linear

Quando os parâmetros da reta são definidos, é possível prever o comportamento do objeto em todo o trajeto.



Como descobrir o conjunto de parâmetros que descrevem melhor a reta?

	s
	2
	8
	14
6	20
8	26

Regressão Linear e Polinomial

- **Problemas de Regressão**

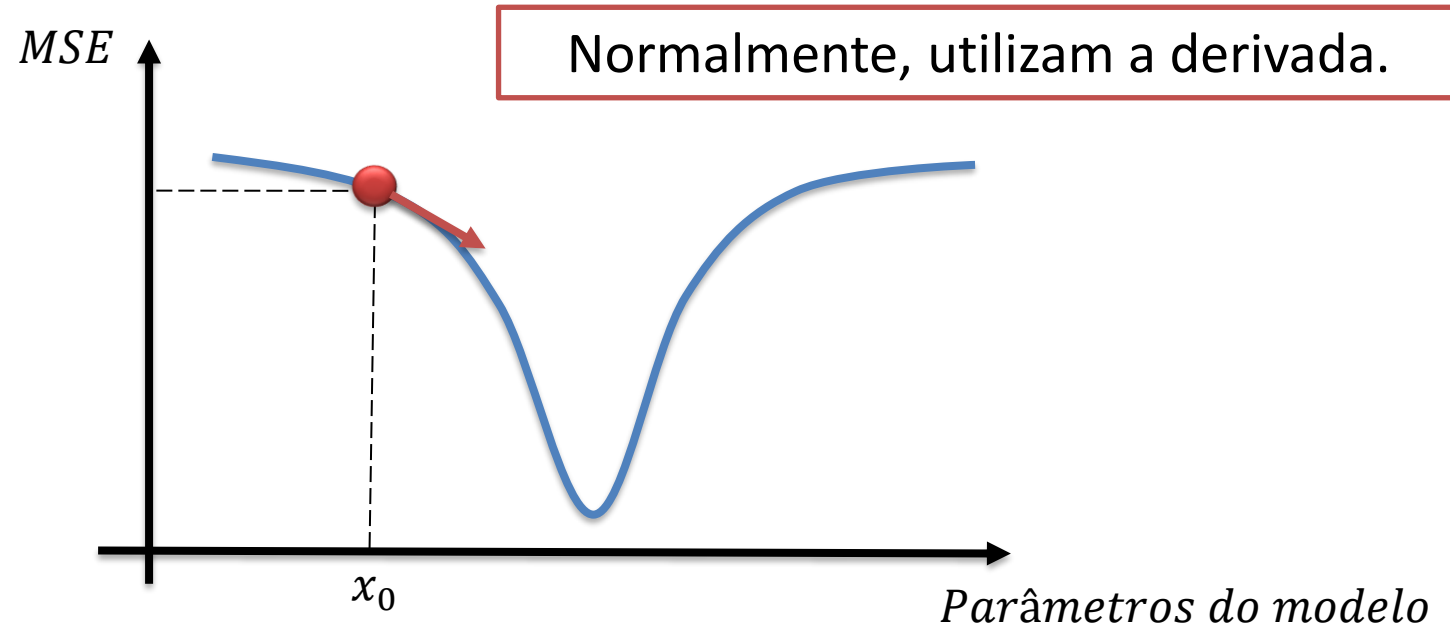
- Para resolver este problema, precisamos interpretá-lo como um **problema de otimização**.
- O **Erro Médio Quadrático** (*MSE - Mean Square Error*) é definido como a soma das distâncias entre os pontos experimentais e os simulados pelo modelo.

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_{exp} - x_{modelo})^2$$

- **Nosso objetivo:** encontrar o modelo que **minimiza** este erro.

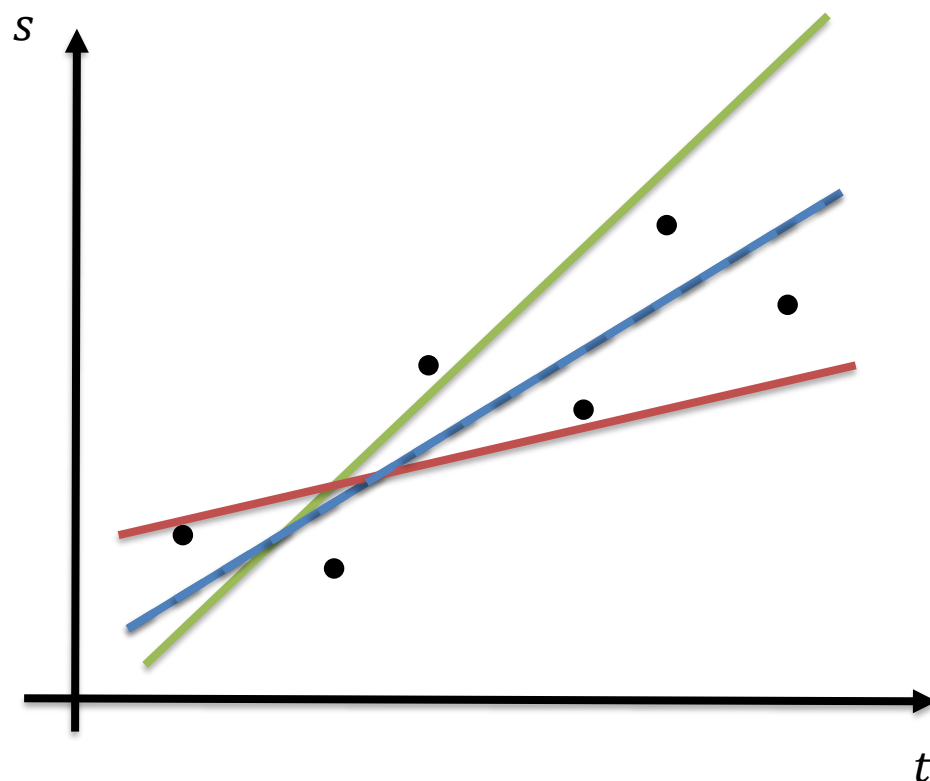
Regressão Linear e Polinomial

- **Problemas de Regressão**
 - Para isso, usamos **Métodos de Otimização**.
 - Esses métodos são utilizados para fazer a melhor **estimação de parâmetros** para um modelo, **minimizando o erro**.



Regressão Linear e Polinomial

- **Regressão linear**
 - Voltando ao exemplo:



$$s(t) = at + b$$

Três exemplos de possíveis soluções:

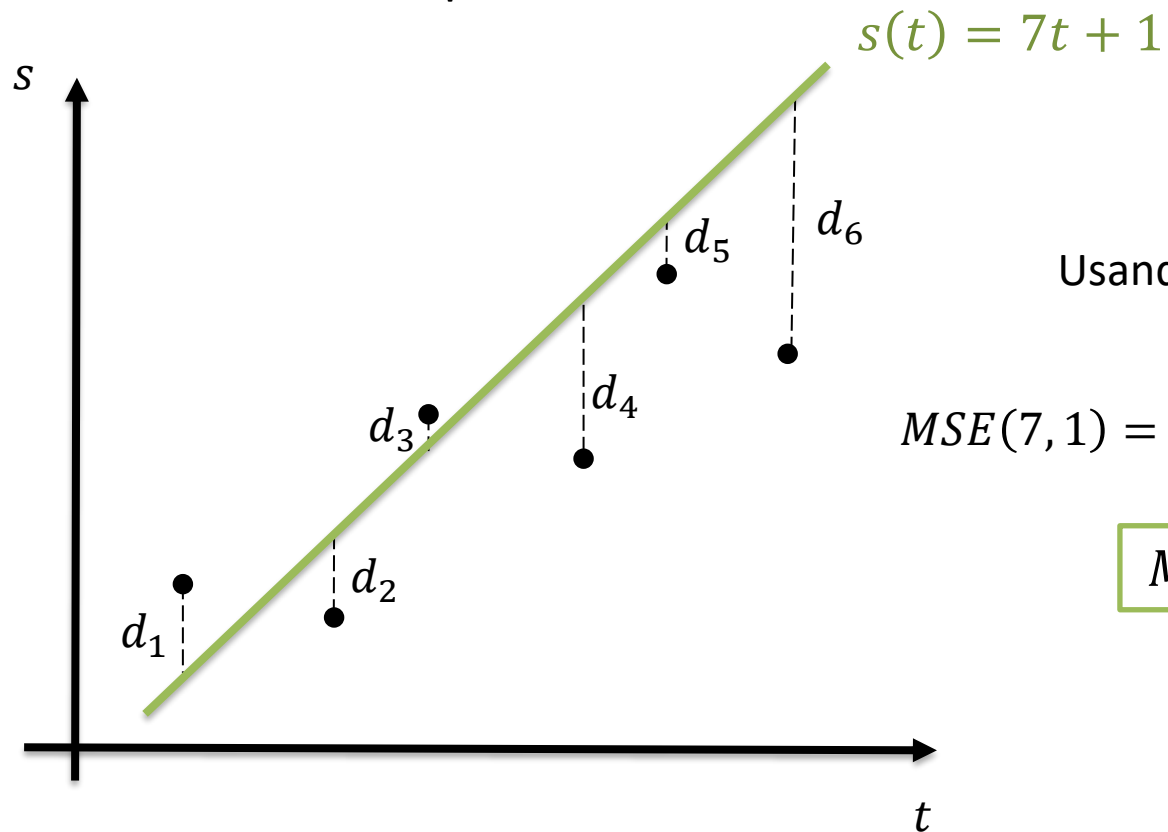
$$s(t) = 7t + 1$$

$$s(t) = 1t + 5$$

$$s(t) = 3t + 2$$

Regressão Linear e Polinomial

- Regressão linear
- Voltando ao exemplo:



Usando a função objetivo:

$$MSE(7, 1) = \frac{d_1^2 + d_2^2 + d_3^2 + d_4^2 + d_5^2 + d_6^2}{6}$$

$$MSE(7, 1) = 3,33$$

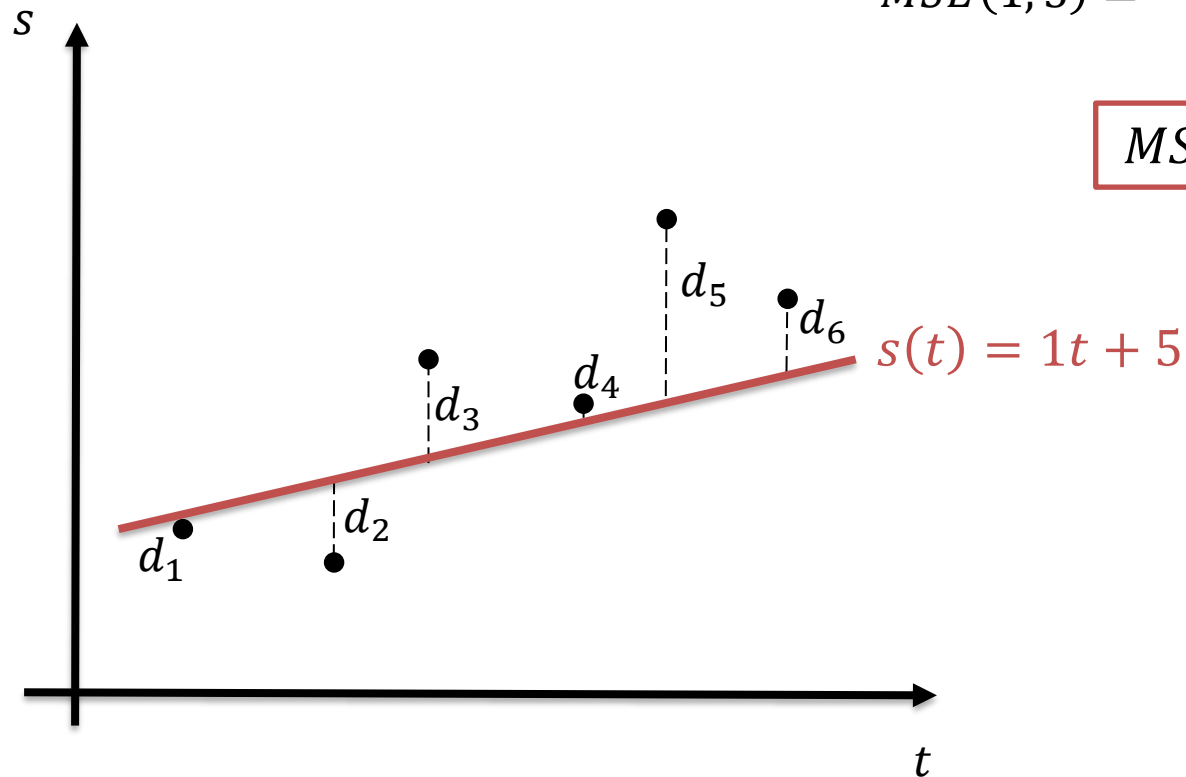
Regressão Linear e Polinomial

$$MSE(7, 1) = 3,33$$

- Regressão linear
- Voltando ao exemplo:

$$MSE(1, 5) = \frac{d_1^2 + d_2^2 + d_3^2 + d_4^2 + d_5^2 + d_6^2}{6}$$

$$MSE(1, 5) = 0,88$$

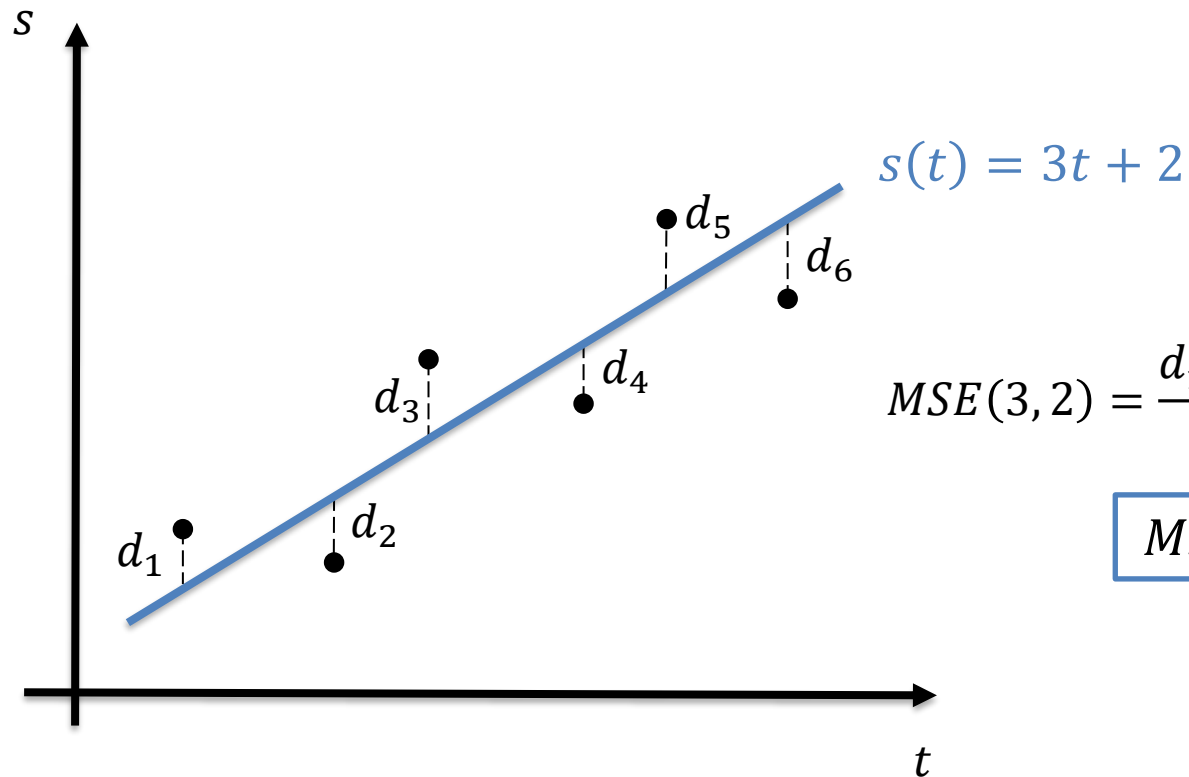


Regressão Linear e Polinomial

- **Regressão linear**
- Voltando ao exemplo:

$$MSE(7, 1) = 3,33$$

$$MSE(1, 5) = 0,88$$

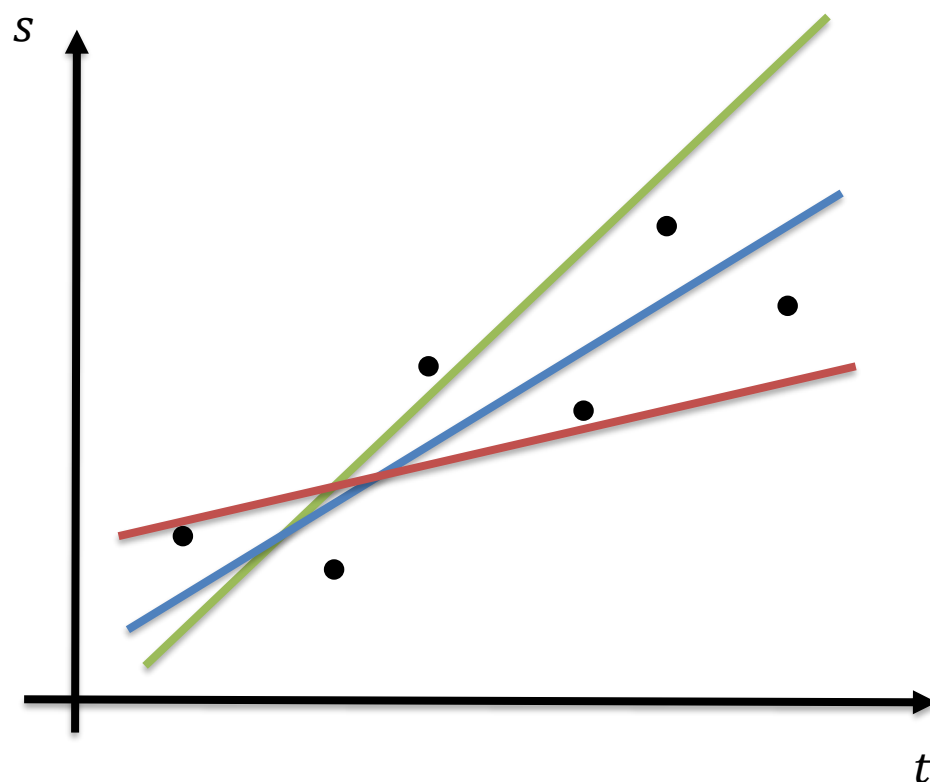


$$MSE(3, 2) = \frac{d_1^2 + d_2^2 + d_3^2 + d_4^2 + d_5^2 + d_6^2}{6}$$

$$MSE(3, 2) = 0,3$$

Regressão Linear e Polinomial

- Regressão linear
 - Voltando ao exemplo:



$$MSE(7, 1) = 3,33$$

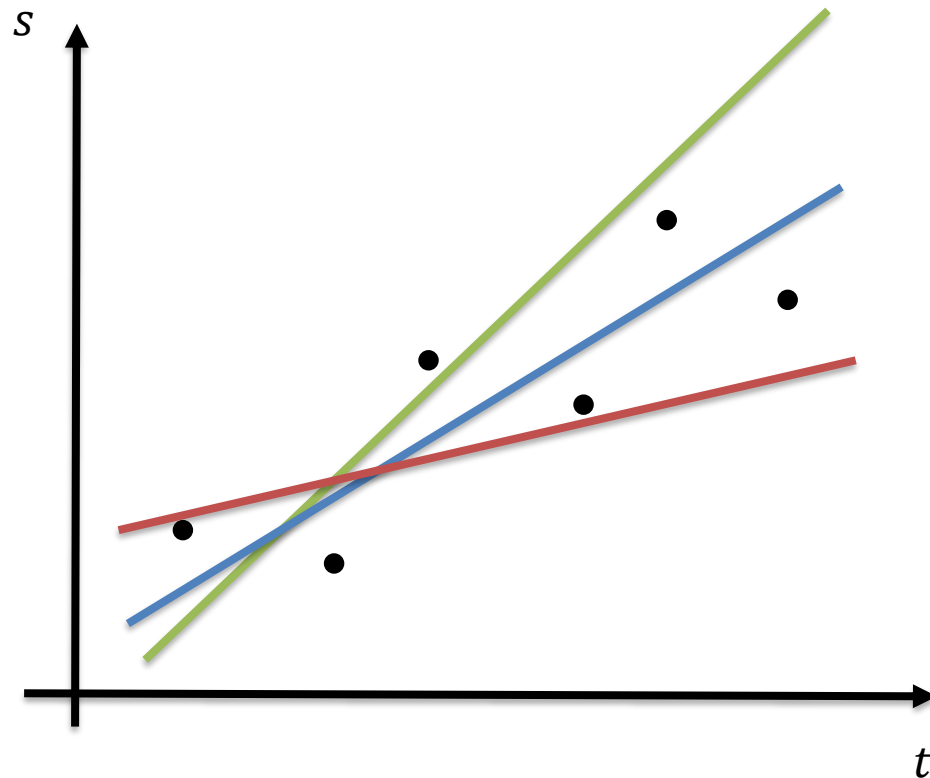
$$MSE(1, 5) = 0,88$$

$$MSE(3, 2) = 0,3$$

Qual foi o melhor modelo?

Regressão Linear e Polinomial

- Regressão linear
- Voltando ao exemplo:



$$MSE(7, 1) = 3,33$$

$$MSE(1, 5) = 0,88$$

$$MSE(3, 2) = 0,3$$

Solução vencedora!

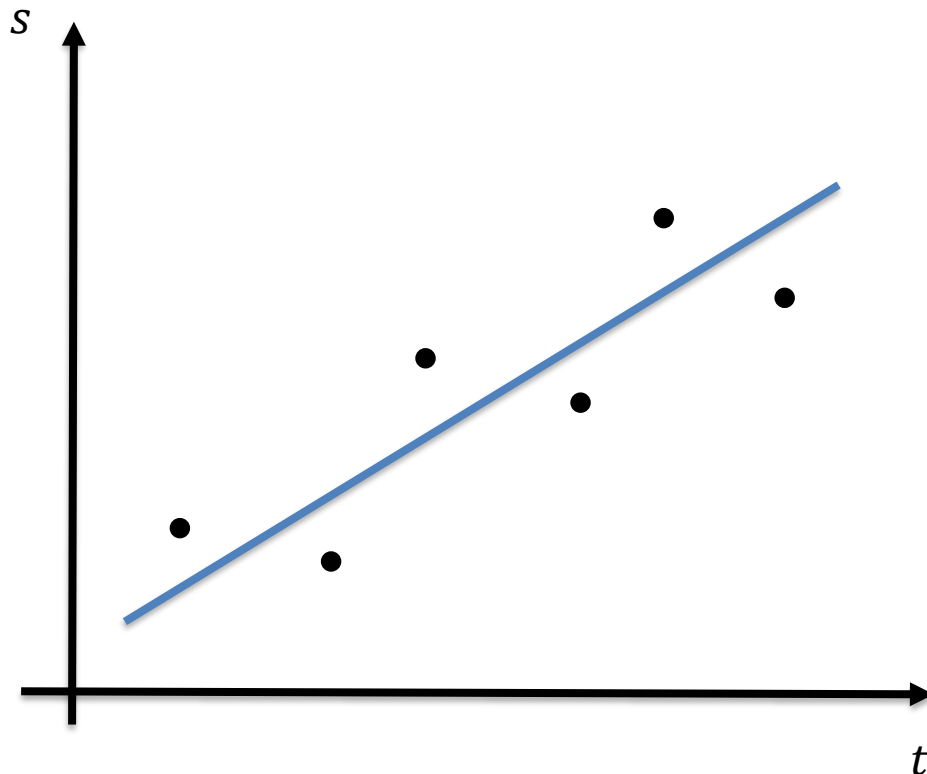
$$a = 3$$

$$b = 2$$

$$s(t) = 3t + 2$$

Regressão Linear e Polinomial

- **Regressão linear**
- Voltando ao exemplo:



Neste caso,
conseguimos uma boa
solução porque o
comportamento de um
objeto em **movimento
linear** segue um **modelo
linear**!

$$y = ax + b$$

$$s(t) = vt + s_0$$

$$s(t) = 3t + 2$$

- **Regressão linear**

- **Exemplo prático:** vamos usar a base “salary_data.csv”, que descreve uma variação salarial de acordo com a idade.

- O primeiro passo é fazer a leitura e pré-processamento da base:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

base = pd.read_csv('salary_data.csv')

# Separando dados em previsores e classes
cols_previsores = ['YearsExperience']

cols_objetivo = ['Salary']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# Separando em base de testes e treinamento (usando 25% para teste)
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
                                                                                                    objetivo,
                                                                                                    test_size=0.25,
                                                                                                    random_state=0)

# Visualização dos dados
plt.scatter(previsores, objetivo)
plt.title('Regressão linear (dados completos)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')
```


- **Regressão linear**

- **Exemplo prático:** varia
variação salarial de

- O primeiro passo é f

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np
```

```
##### Preprocessamento
```

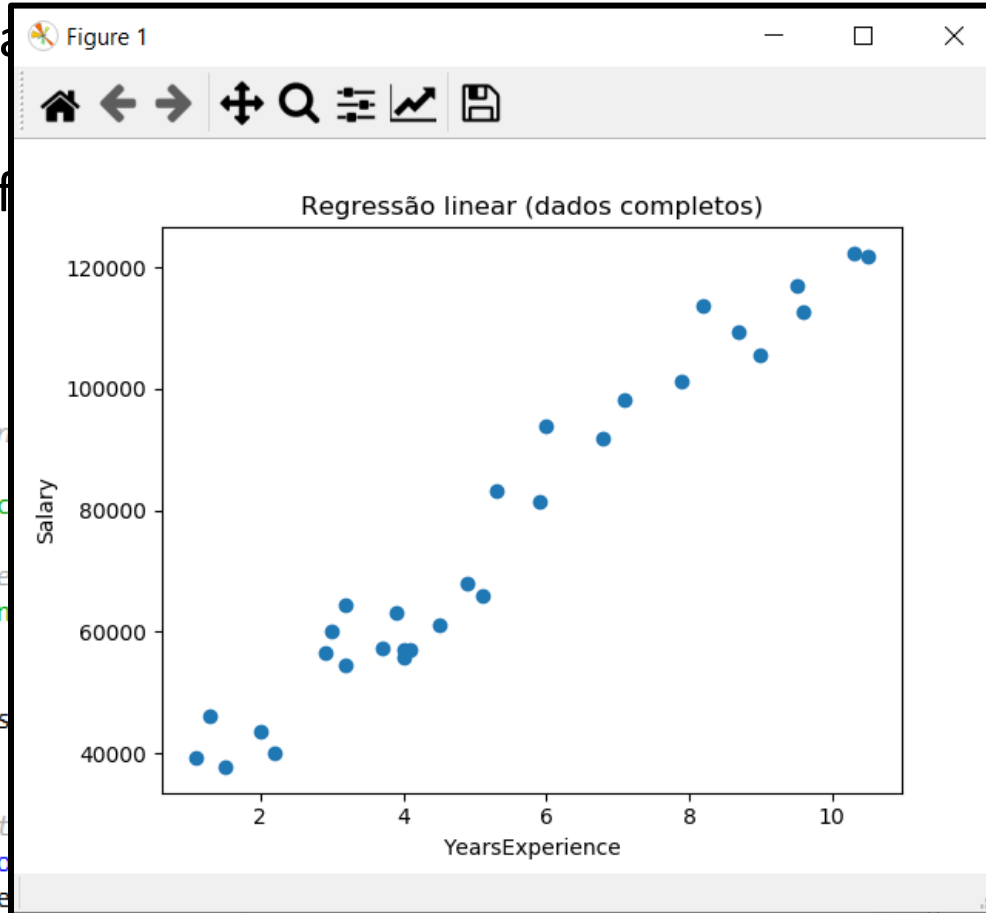
```
base = pd.read_csv('salary_data.csv')
```

```
# Separando dados em previsores e
cols_previsores = ['YearsExperience']
```

```
cols_objetivo = ['Salary']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]
```

```
# Separando em base de testes e tre
from sklearn.model_selection import
previsores_treinamento, previsores
```

```
# Visualização dos dados
plt.scatter(previsores, objetivo)
plt.title('Regressão linear (dados completos)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')
```



escreve uma

e:

```
it(previsores,
    objetivo,
    test_size=0.25,
    random_state=0)
```

- **Regressão linear**

- Em seguida, a Regressão Linear é feita utilizando-se a classe LinearRegression:

```
##### Regressão Linear #####  
  
from sklearn.linear_model import LinearRegression  
regressor = LinearRegression()  
  
# Treinamento  
regressor.fit(previsores_treinamento, objetivo_treinamento)  
  
# Teste  
previsoes = regressor.predict(previsores_teste)
```

- **Regressão linear**

- Em seguida, a Regressão Linear é feita utilizando-se a classe LinearRegression:

```
##### Regressão Linear #####
```

```
from sklearn.linear_model import LinearRegression  
regressor = LinearRegression()
```

```
# Treinamento
```

```
regressor.fit(previsores_treinamento, objetivo_treinamento)
```

```
# Teste
```

```
previsoes = regressor.predict(previsores_teste)
```



O comando “fit” realiza o ajuste da reta para os dados de treinamento.

- **Regressão linear**

- Em seguida, a Regressão Linear é feita utilizando-se a classe `LinearRegression`:

```
##### Regressão Linear #####
```

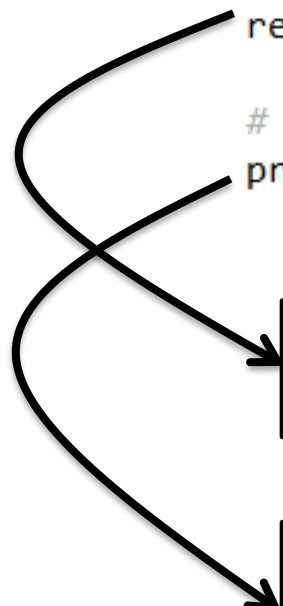
```
from sklearn.linear_model import LinearRegression  
regressor = LinearRegression()
```

```
# Treinamento
```

```
regressor.fit(previsores_treinamento, objetivo_treinamento)
```

```
# Teste
```

```
previsoes = regressor.predict(previsores_teste)
```



O comando “fit” realiza o ajuste da reta para os dados de treinamento.

Depois, o método “predict” irá calcular a previsão para os dados previsores de teste, usando a reta ajustada.

- **Regressão linear**

- Por fim, podemos avaliar a qualidade do modelo ajustado:

```
##### Avaliação dos resultados #####

# Visualização dos dados de treinamento
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(previsores_treinamento, regressor.predict(previsores_treinamento), color = 'red')
plt.title('Regressão linear (treinamento)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')

# Visualização dos dados de teste
plt.scatter(previsores_teste, objetivo_teste)
plt.plot(previsores_teste, previsoes, color = 'red')
plt.title('Regressão linear (teste)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')

score = regressor.score(previsores_teste, objetivo_teste)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)

coef_0 = regressor.intercept_
coeficientes = regressor.coef_
```

- **Regressão linear**

- Por fim, podemos avaliar a qualidade do modelo ajustado:

```
##### Avaliação dos resultados #####
```

```
# Visualização dos dados de treinamento
```

```
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(previsores_treinamento, regressor.predict(previsores_treinamento), color = 'red')
plt.title('Regressão linear (treinamento)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')
```

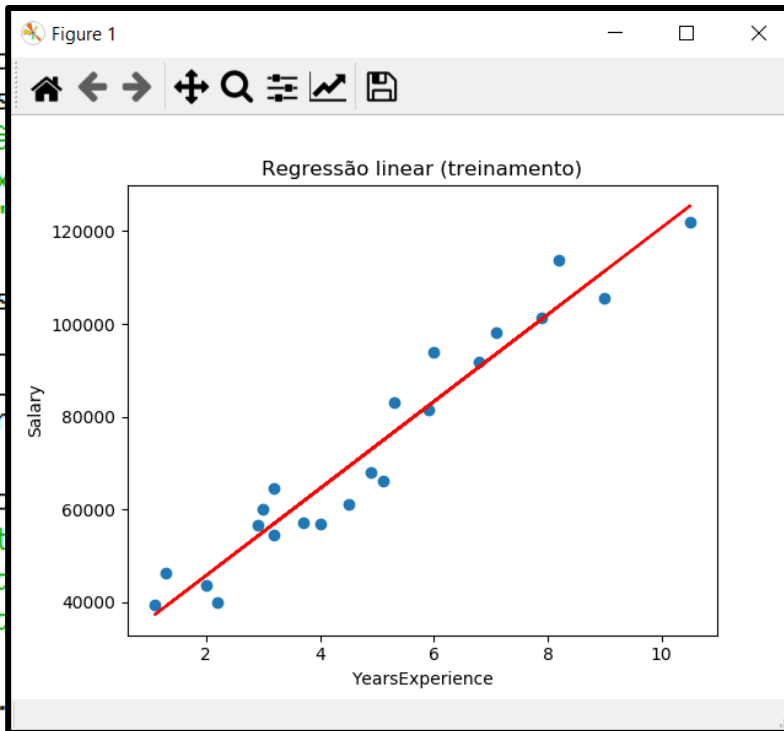
```
# Visualização dos
```

```
plt.scatter(previsores
plt.plot(previsores
plt.title('Regressã
plt.xlabel('YearsEx
plt.ylabel('Salary'
```

```
score = regressor.s
mae = metrics.mean
mse = metrics.mean
rmse = np.sqrt(metr
```

```
print('Score:', sco
print('Mean Absolut
print('Mean Squared
print('Root Mean So
```

```
coef_0 = regressor.
coeficientes = regressor.coef_
```



soes))

- **Regressão linear**

- Por fim, podemos avaliar a qualidade do modelo ajustado:

```
##### Avaliação dos resultados #####
```

```
# Visualização dos dados de treinamento
```

```
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(previsores_treinamento, regressor.predict(previsores_treinamento), color = 'red')
plt.title('Regressão linear (treinamento)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')
```

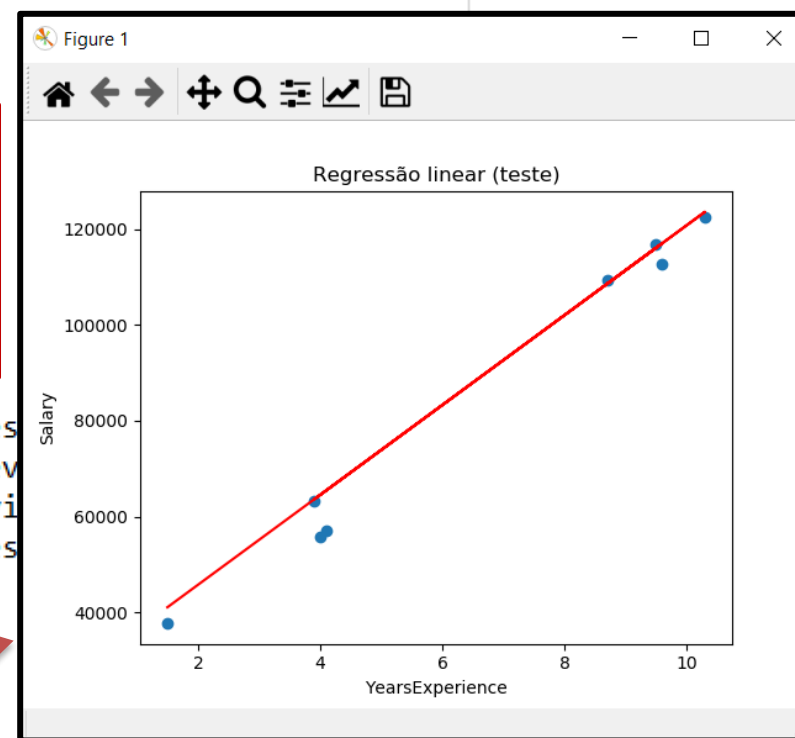
```
# Visualização dos dados de teste
```

```
plt.scatter(previsores_teste, objetivo_teste)
plt.plot(previsores_teste, previsoes, color = 'red')
plt.title('Regressão linear (teste)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')
```

```
score = regressor.score(previsores_teste, objetivo_teste)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))
```

```
print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

```
coef_0 = regressor.intercept_
coeficientes = regressor.coef_
```

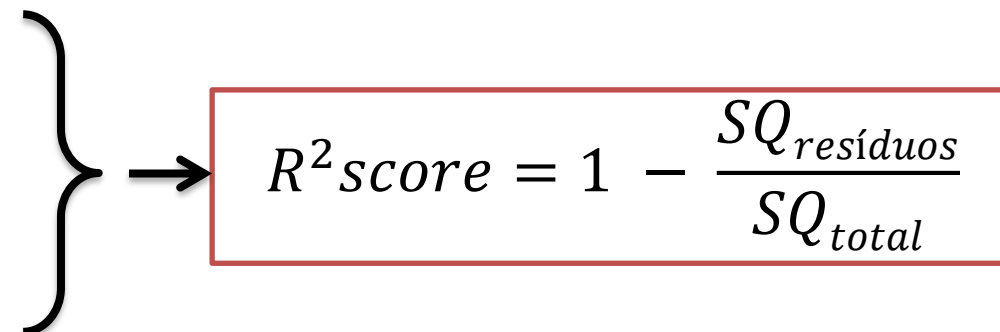


Regressão Linear e Polinomial

- **Coeficiente de Determinação (R^2 score)**
 - Esta métrica mede a capacidade do modelo de **explicar os dados**.
 - Possui valor máximo igual a 1, o que significaria que as variáveis independentes explicam 100% do comportamento da variável objetivo.
 - O R^2 score é calculado da seguinte maneira:

$$SQ_{resíduos} = \sum_{i=1}^N (x_{exp} - x_{modelo})^2$$

$$SQ_{total} = \sum_{i=1}^N (x_{exp} - x_{média})^2$$


$$R^2 score = 1 - \frac{SQ_{resíduos}}{SQ_{total}}$$

- **Regressão linear**

- Por fim, podemos avaliar a qualidade do modelo ajustado:

```
##### Avaliação dos resultados #####
```

```
# Visualização dos dados de treinamento
```

```
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(previsores_treinamento, regressor.predict(previsores_treinamento), color = 'red')
plt.title('Regressão linear (treinamento)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')
```

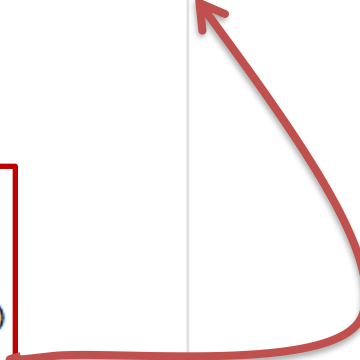
```
# Visualização dos dados de teste
```

```
plt.scatter(previsores_teste, objetivo_teste)
plt.plot(previsores_teste, previsoes, color = 'red')
plt.title('Regressão linear (teste)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')
```

```
score = regressor.score(previsores_teste, objetivo_teste)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

```
coef_0 = regressor.intercept_
coeficientes = regressor.coef_
```



Score: 0.9779208335417602
Mean Absolute Error: 3508.5455930660555
Mean Squared Error: 22407940.143340684
Root Mean Squared Error: 4733.70258289858

- **Regressão linear**

- Por fim, podemos avaliar a qualidade do modelo ajustado:

```
##### Avaliação dos resultados #####
```

```
# Visualização dos dados de treinamento
```

```
plt.scatter(previsores_treinamento, objetivo_treinamento)
plt.plot(previsores_treinamento, regressor.predict(previsores_treinamento), color = 'red')
plt.title('Regressão linear (treinamento)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')
```

```
# Visualização dos dados de teste
```

```
plt.scatter(previsores_teste, objetivo_teste)
plt.plot(previsores_teste, previsoes, color = 'red')
plt.title('Regressão linear (teste)')
plt.xlabel('YearsExperience')
plt.ylabel('Salary')
```

```
score = regressor.score(previsores_teste, objetivo_teste)
```

```
mae = metrics.mean_absolute_error(y_test, y_hat)
```

```
mse = metrics.mean_squared_error(y_test, y_hat)
```

```
rmse = np.sqrt(metrics.mean_squared_error(y_test, y_hat))
```

```
print('Score:', score)
```

```
print('Mean Absolute Error:', mae)
```

```
print('Mean Squared Error:', mse)
```

```
print('Root Mean Squared Error:', rmse)
```

```
coef_0 = regressor.intercept_
```

```
coeficientes = regressor.coef_
```

coef_0	Array of float64	(1,)	[26986.69131674]
coeficientes	Array of float64	(1, 1)	[[9379.71049195]]

$$y = ax + b$$

Regressão Linear e Polinomial

- **Regressão linear múltipla**

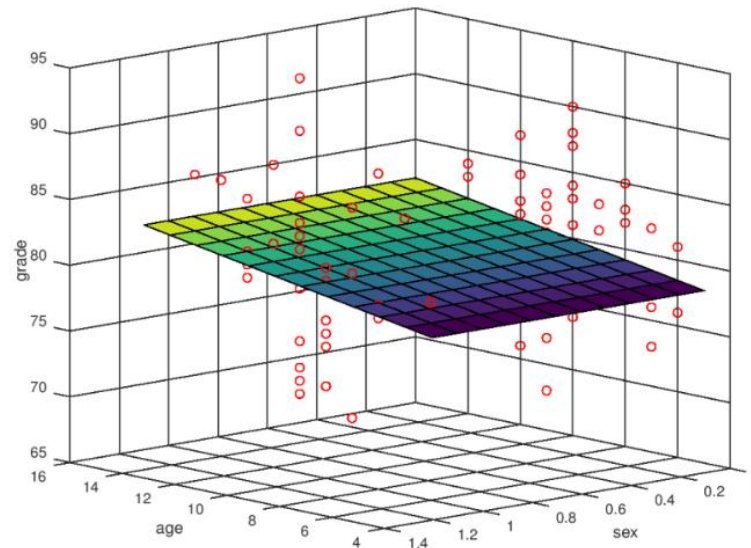
- A maior parte dos problemas reais envolve **mais de uma variável previsora**.
- Nesses casos, podemos fazer a chamada “**Regressão Linear Múltipla**”.
- O modelo, desta vez, é um **hiperplano**, dado pela seguinte equação:

$$y = a_0 + b_1x_1 + b_2x_2 + b_3x_3 + \cdots + b_nx_n$$

- Onde os **valores de x** correspondem às **variáveis previsoras** do problema.

Regressão Linear e Polinomial

- **Regressão linear múltipla**
 - Este tipo de modelo será eficiente para os casos em que as variáveis previsoras são **linearmente independentes**.
 - Além disso, o comportamento de cada variável com relação à variável objetivo também deve ser **linear**.



Regressão Linear e Polinomial

- **Regressão linear múltipla**
 - Como este método é capaz de resolver problemas reais, vamos utilizar uma **base real** como exemplo.
 - A base que vamos utilizar possui os **preços de imóveis** em um distrito dos EUA e as **características do imóvel**.
 - O objetivo é **ajustar um modelo** que permita a **previsão dos valores** dos imóveis de acordo com as variáveis previsoras.



<https://www.kaggle.com/harlfoxem/housesalesprediction>

- **Regressão linear**

- O primeiro passo é ler e fazer o pré-processamento dos dados.

```
import pandas as pd

##### #Pré-processamento dos dados #####

#Leitura dos dados
base = pd.read_csv('house_prices.csv')
base.describe()

# =====
#                               Tratando valores inválidos
# =====
# Não há valores inconsistentes

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# =====
#                               Separando dados em previsores e objetivo
# =====

cols_previsores = ['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
                   'floors', 'waterfront', 'view', 'condition', 'grade', 'sqft_above',
                   'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode', 'lat', 'long']
# Não usei: date, sqft_living15 e sqft_lot15

cols_objetivo = ['price']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# =====
#                               Transformar as variáveis categóricas em valores numéricos
# =====
# Todas as variáveis são numéricas

# =====
#                               Separando em base de testes e treinamento
# =====

# usando 25% para teste
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
                                                                                                  objetivo,
                                                                                                  test_size=0.25,
                                                                                                  random_state=0)

# =====
#                               Padronização dos dados
# =====

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
previsores_treinamento = scaler.fit_transform(previsores_treinamento)
previsores_teste = scaler.transform(previsores_teste)
```

- **Regressão linear**

- Para fazer a regressão linear múltipla, faça o **mesmo procedimento** que fizemos para a regressão linear simples.

- O LinearRegression será capaz de interpretar sozinha que deve utilizar todas as variáveis previsoras para **ajustar o modelo** linear.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

# Arquivo separado

##### Regressão Linear Múltipla #####

from sklearn.linear_model import LinearRegression
regressor = LinearRegression()

# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(previsores_teste)

##### Avaliação dos resultados #####

score = regressor.score(previsores_teste, objetivo_teste)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)

# Parâmetros estimados para o modelo
coef_0 = regressor.intercept_
coeficientes = regressor.coef_
```

- **Regressão linear**

- Para fazer a regressão linear múltipla, faça o **mesmo procedimento** que fizemos para a regressão linear simples.

- O LinearRegression será capaz de interpretar que deve ajustar as variáveis previsoras para ajustar o modelo linear.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

# Arquivo separado
```

Perceba que, além do coeficiente 0, foram ajustados 16 coeficientes, um para cada variável previsoras.

$$y = a_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

```
previsores = regressor.predict(previsores_teste)

##### Avaliação dos resultados #####

score = regressor.score(previsores_teste, objetivo_teste)

print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

coef_0	Array of float64 (1,)	[538742.81487137]
coeficientes	Array of float64 (1, 16)	[[-2.89422513e+04 2.68367521e+04 3.15454507e+18 ... -3.05835426e+04 ...]]

```
# Parâmetros estimados para o modelo
coef_0 = regressor.intercept_
coeficientes = regressor.coef_
```


- **Regressão**

Vamos registrar os resultados em um arquivo único, para que possamos compará-lo com resultados futuros:

```
Score: 0.6909056569846175
Mean Absolute Error: 123206.03501553473
Mean Squared Error: 41060274233.59005
Root Mean Squared Error: 202633.34926312117
```

```
##
```

- Para fazer linear múltiplo, usamos o mesmo procedimento que fizemos para a

```
##### # Regressão Linear Múltipla #####
```

```
from sklearn.linear_model import LinearRegression
```

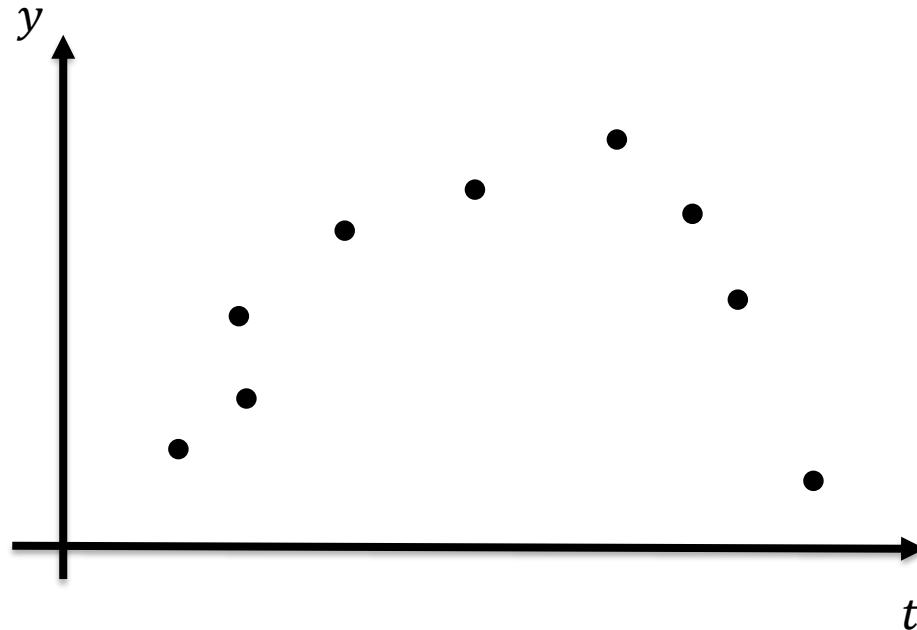
House Sales in King County, USA				
Descrição	Dados e imóveis e seus respectivos valores em um distrito dos Estados Unidos.			
Total de registros:	21613			
Total de características:	16			
Total de colunas objetivo:	1			
Resultados				
Método	Configuração	Tratamento de dados	Score %	RMSE
Regressão Linear Múltipla	-	Padronização	0.69	202633.35

```
coeficientes = regressor.coef_
```

Regressão Linear e Polinomial

- **Regressão polinomial**
 - O modelo linear, mesmo o múltiplo, é bem **simples**.
 - Existem casos em que **outros padrões** de comportamento podem ser observados.

➤ Por exemplo: o lançamento de um projétil.



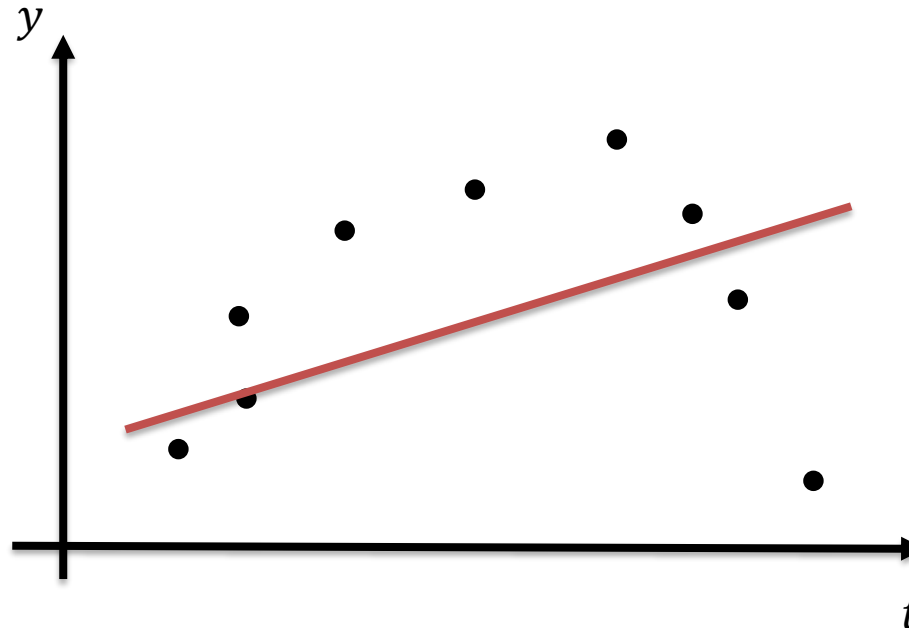
Regressão Linear e Polinomial

- **Regressão polinomial**

- O modelo linear, mesmo o múltiplo, é bem **simples**.
- Existem casos em que **outros padrões** de comportamento podem ser observados.

➤ Por exemplo: o lançamento de um projétil.

- Uma reta não seria capaz de descrever bem o comportamento dos dados.



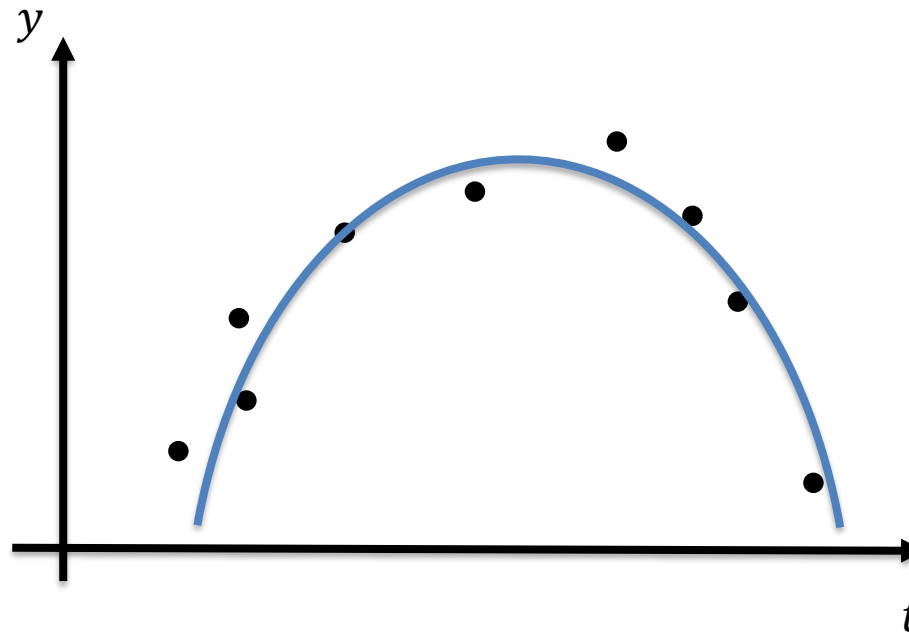
Regressão Linear e Polinomial

- **Regressão polinomial**

- Neste caso, sabemos que **existe um modelo** que descreve corretamente o movimento de um projétil:

$$s(t) = s_0 + v_0 t - \frac{at^2}{2}$$

- O que precisamos, nesse caso, é identificar o valor correto dos parâmetros s_0 , v_0 e a .
- Com estes valores, teremos um modelo que descreve corretamente este movimento!



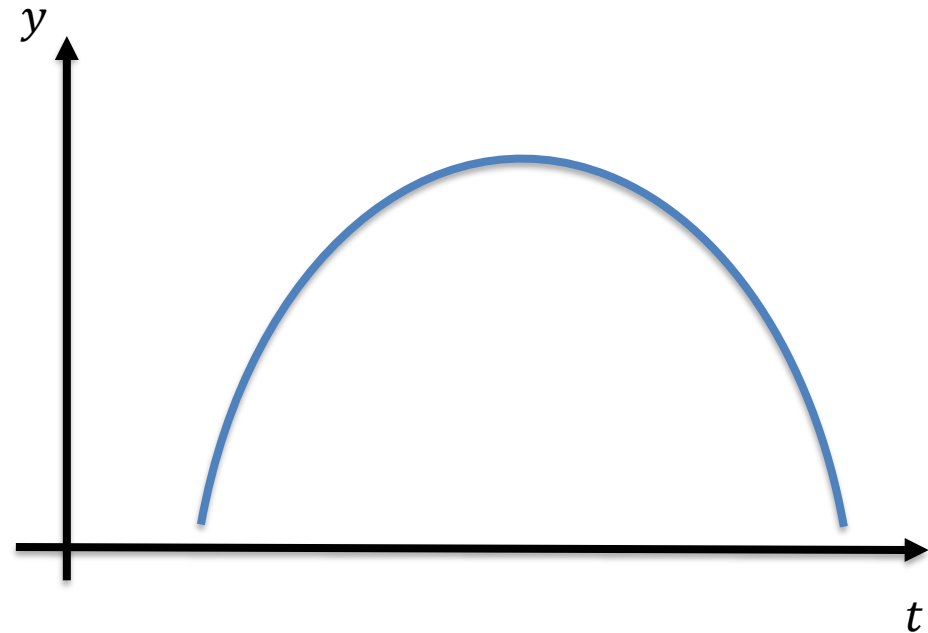
Regressão Linear e Polinomial

- **Regressão polinomial**

- Se temos os valores dos coeficientes, podemos prever os valores para qualquer instante de tempo;

$$y(t) = -5t^2 + 20t + 1$$

t	y



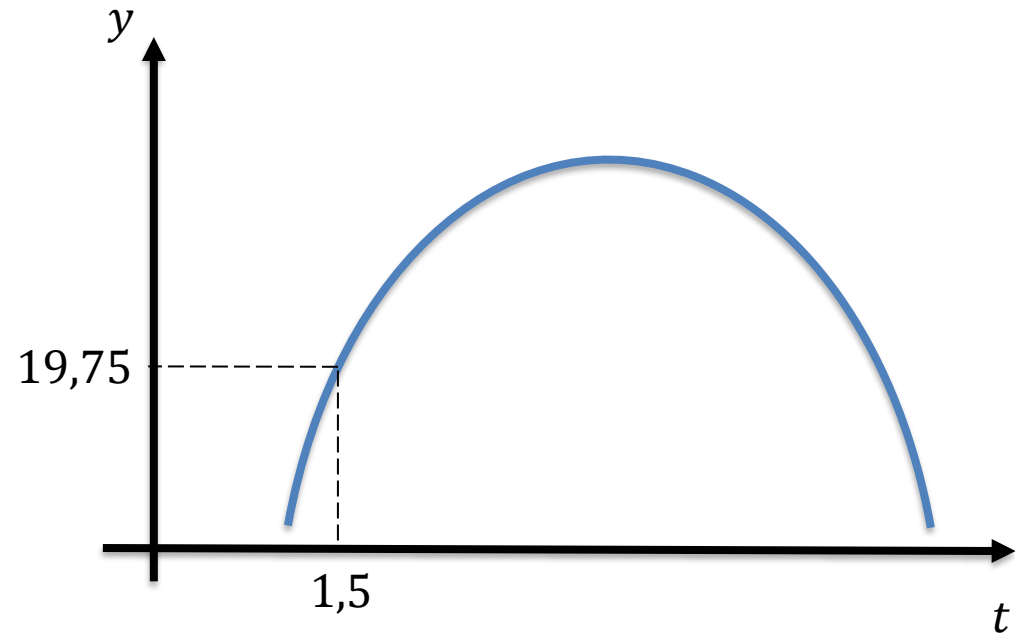
Regressão Linear e Polinomial

- **Regressão polinomial**

- Se temos os valores dos coeficientes, podemos prever os valores para qualquer instante de tempo;

$$y(t) = -5t^2 + 20t + 1$$

t	y
1,5	19,75



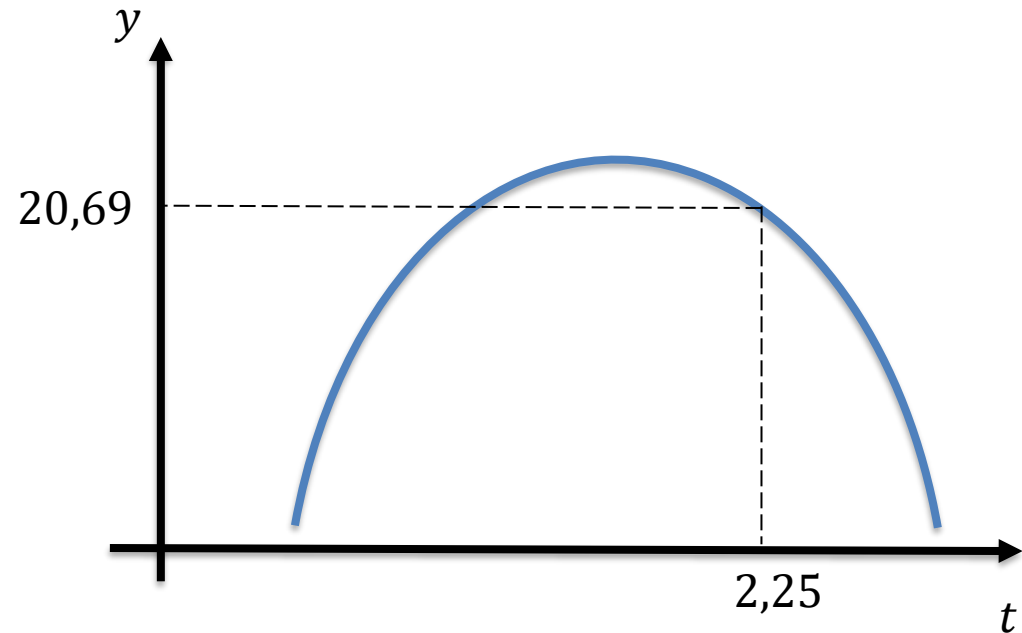
Regressão Linear e Polinomial

- **Regressão polinomial**

- Se temos os valores dos coeficientes, podemos prever os valores para qualquer instante de tempo;

$$y(t) = -5t^2 + 20t + 1$$

t	y
1,5	19,75
2,25	20,69



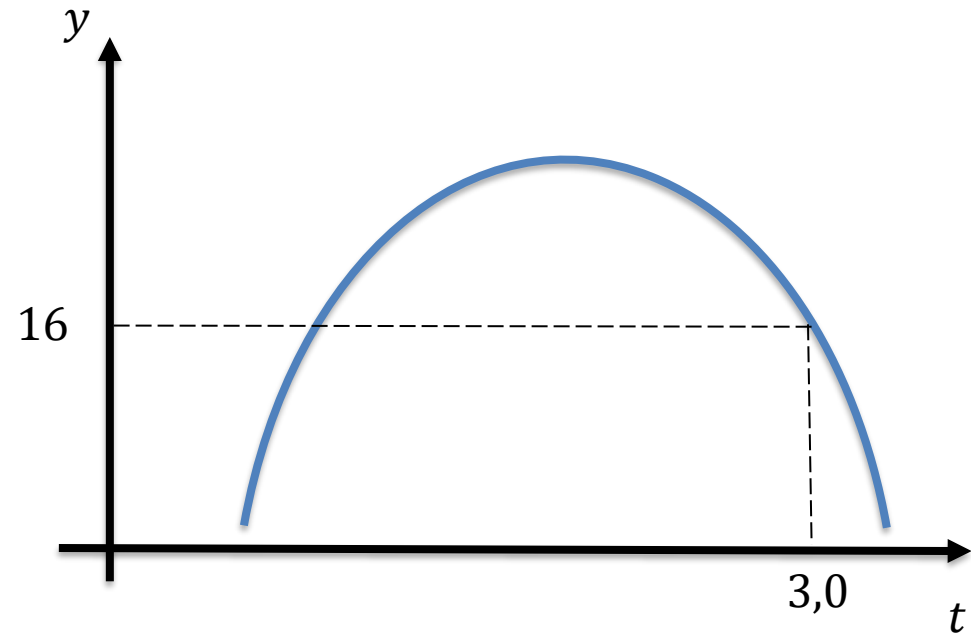
Regressão Linear e Polinomial

- **Regressão polinomial**

- Se temos os valores dos coeficientes, podemos prever os valores para qualquer instante de tempo;

$$y(t) = -5t^2 + 20t + 1$$

t	y
1,5	19,75
2,25	20,69
3,0	16



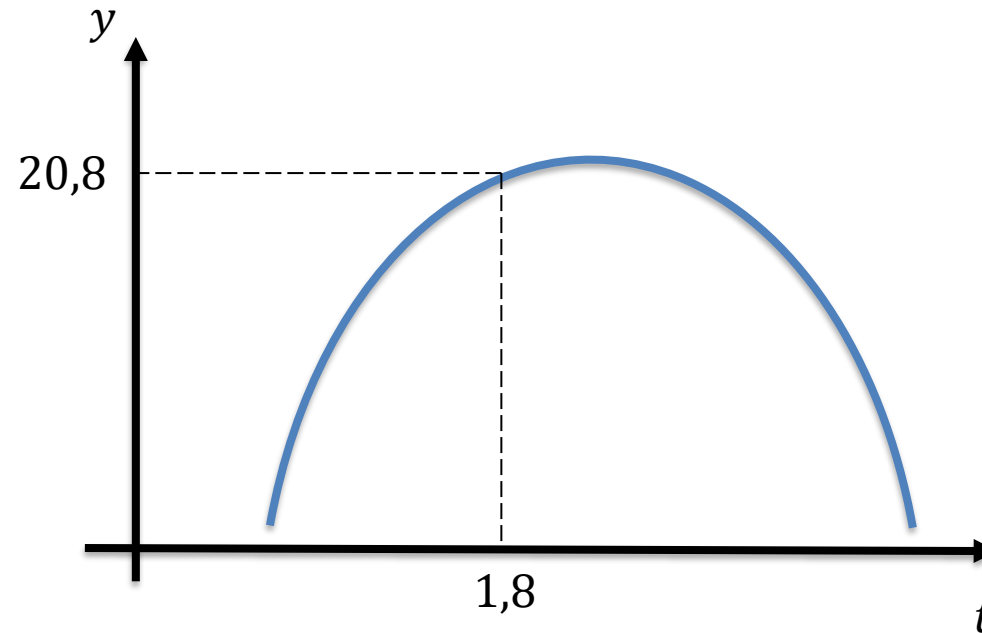
Regressão Linear e Polinomial

- **Regressão polinomial**

- Se temos os valores dos coeficientes, podemos prever os valores para qualquer instante de tempo;

$$y(t) = -5t^2 + 20t + 1$$

t	y
1,5	19,75
2,25	20,69
3,0	16
1,8	20,8



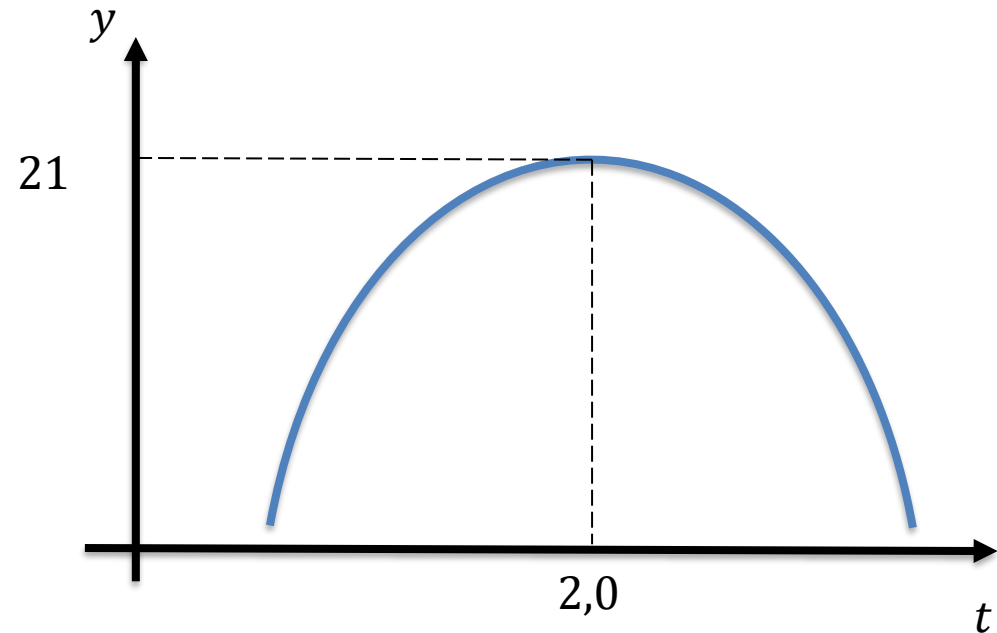
Regressão Linear e Polinomial

- **Regressão polinomial**

- Se temos os valores dos coeficientes, podemos prever os valores para qualquer instante de tempo;

$$y(t) = -5t^2 + 20t + 1$$

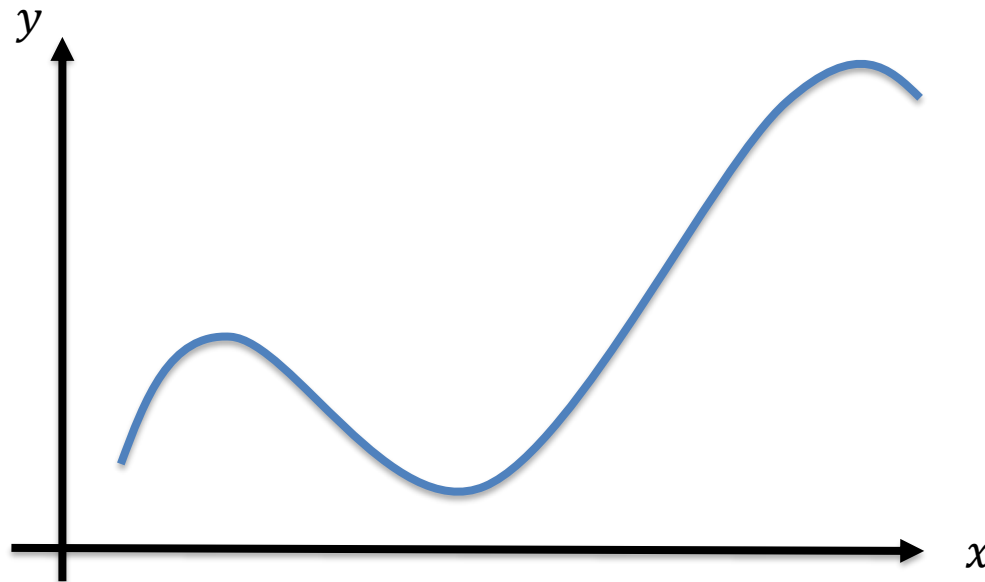
t	y
1,5	19,75
2,25	20,69
3,0	16
1,8	20,8
2,0	21



Regressão Linear e Polinomial

- **Regressão polinomial**
 - A **Regressão Polinomial** permite que façamos um ajuste dos dados a um polinômio de **qualquer grau**, seguindo o modelo:

$$y = a_0 + b_1x_1 + b_2x_1^2 + b_3x_1^3 + \dots + b_nx_1^n + \dots$$



- **Regressão polinomial**

- Vamos analisar uma nova base de dados, referentes ao valor de um plano de saúde de acordo com a idade do cliente:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np

##### Preprocessamento #####

base = pd.read_csv('plano_saude.csv')

# Separando dados em previsores e classes
cols_previsores = ['idade']

cols_objetivo = ['custo']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# Separando em base de testes e treinamento (usando 25% para teste)
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
                                                                                                    objetivo,
                                                                                                    test_size=0.25,
                                                                                                    random_state=0)

# Visualização dos dados
plt.scatter(previsores, objetivo)
plt.title('Regressão linear (dados completos)')
plt.xlabel('idade')
plt.ylabel('custo')
```

- **Regressão polinomial**

- Vamos analisar um plano de saúde de acordo

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import metrics
import numpy as np
```

```
##### Preprocessamento
```

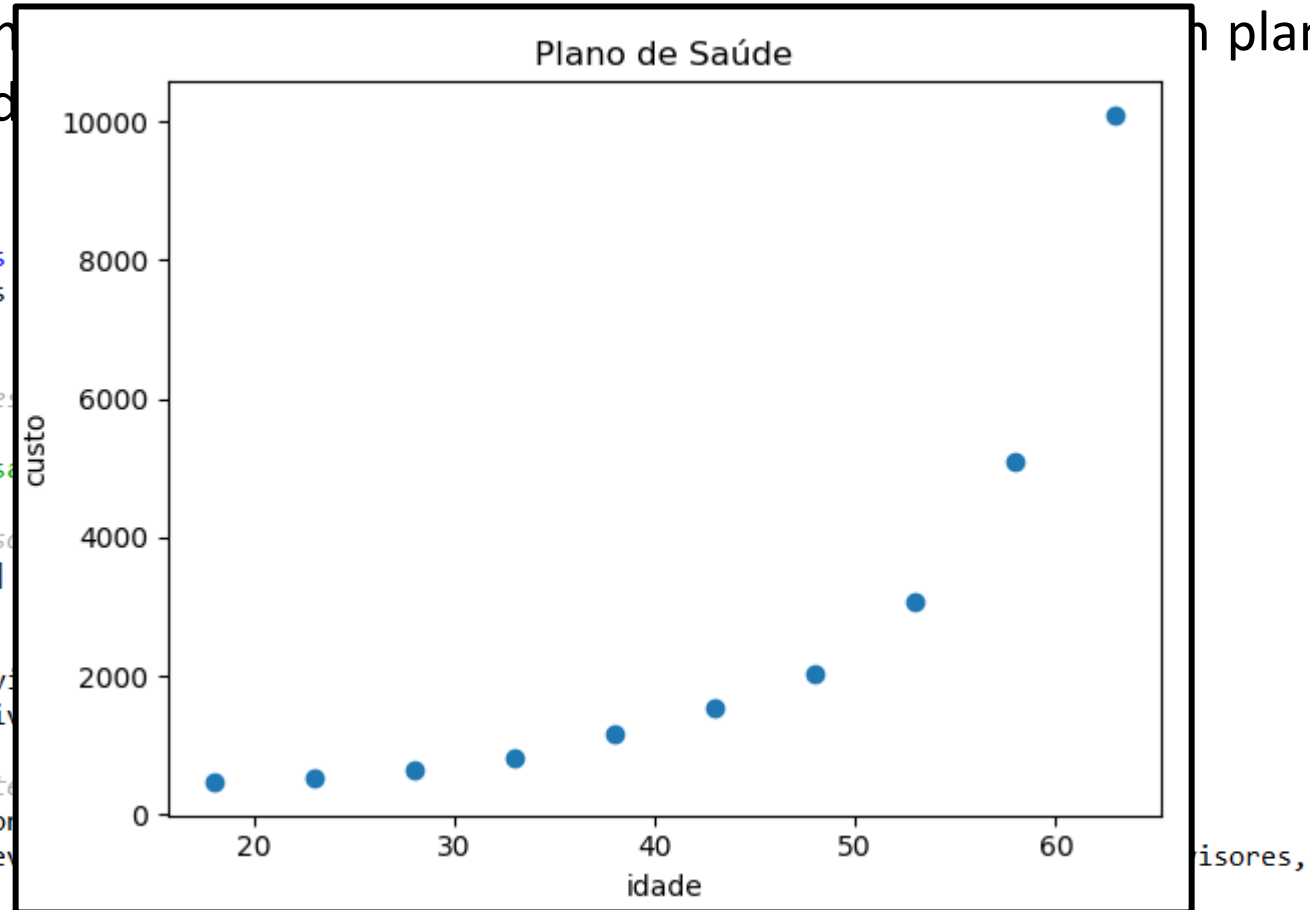
```
base = pd.read_csv('plano_saude.csv')
```

```
# Separando dados em previsores
cols_previsores = ['idade']
```

```
cols_objetivo = ['custo']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]
```

```
# Separando em base de teste
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores, objetivo, test_size=0.25, random_state=0)
```

```
# Visualização dos dados
plt.scatter(previsores, objetivo)
plt.title('Regressão linear (dados completos)')
plt.xlabel('idade')
plt.ylabel('custo')
```



test_size=0.25,
random_state=0)

- **Regressão polinomial**

- Vemos que, aplicando o script de Regressão Linear, o resultado não é satisfatório:

```
##### Regressão Linear #####

from sklearn.linear_model import LinearRegression
regressor = LinearRegression()

# Treinamento
regressor.fit(previsores_treinamento, objetivo_treinamento)

# Teste
previsoes = regressor.predict(previsores_teste)

##### Avaliação dos resultados #####

# Visualização dos dados de treinamento
plt.scatter(previsores, objetivo)
plt.plot(previsores, regressor.predict(previsores), color = 'red')
plt.title('Regressão linear')
plt.xlabel('idade')
plt.ylabel('custo')
```

- **Regressão polinomial**

- Vemos que, aplicando o script de Regressão Linear, o resultado não é satisfatório:

```
#####
```

```
from sklearn.l  
regressor = Li
```

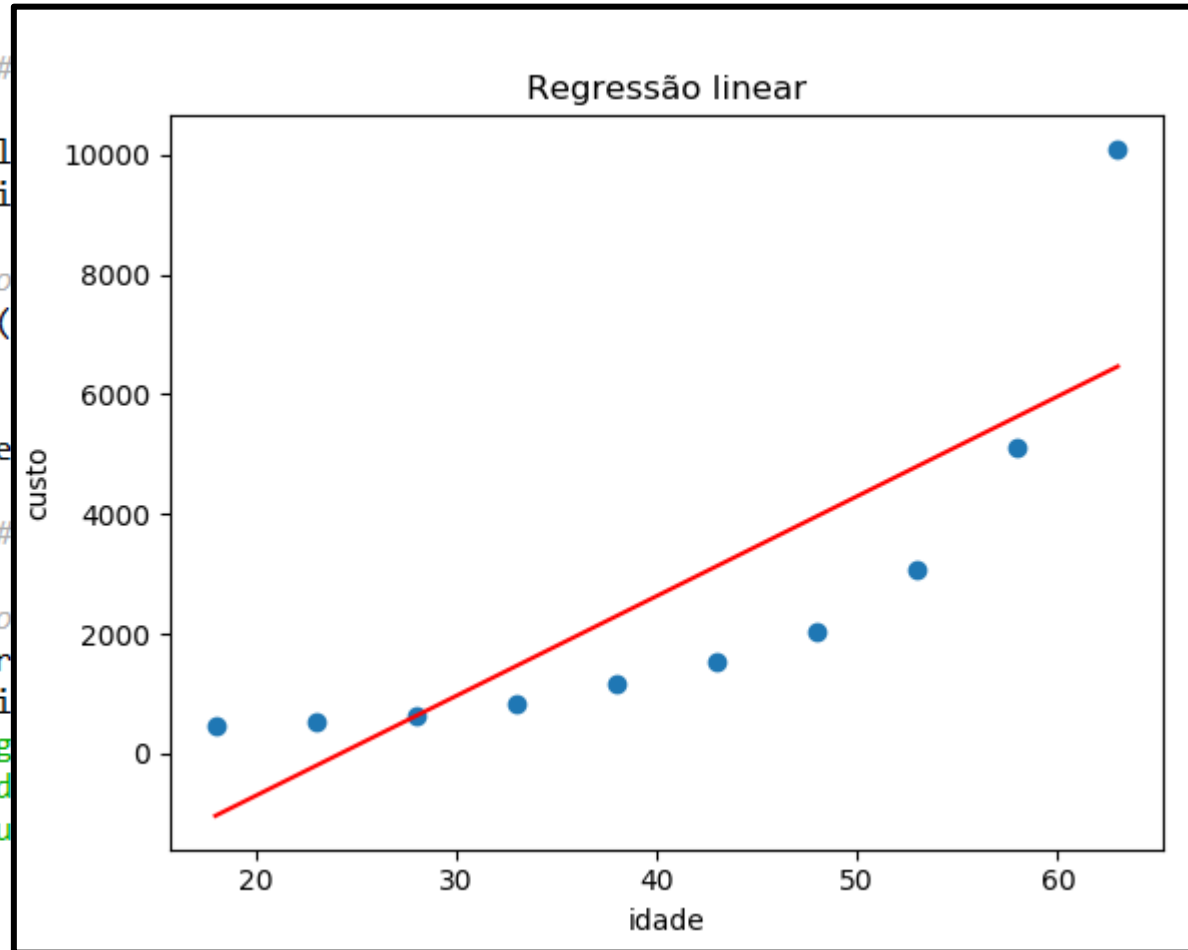
```
# Treinamento  
regressor.fit()
```

```
# Teste  
previsoes = re
```

```
#####
```

```
# Visualização
```

```
plt.scatter(pr  
plt.plot(previsoes, 'r')  
plt.title('Reg  
plt.xlabel('id  
plt.ylabel('cu
```



- **Regressão polinomial**

- Para aplicar a Regressão Polinomial, devemos fazer:

```
##### Regressão Polinomial #####
```

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree = 2)
previsores_treinamento_poly = poly.fit_transform(previsores_treinamento)
previsores_teste_poly = poly.transform(previsores_teste)
```

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
```

```
# Treinamento
regressor.fit(previsores_treinamento_poly, objetivo_treinamento)
```

```
# Teste
previsoes = regressor.predict(previsores_teste_poly)
```


- **Regressão polinomial**

- Para aplicar a Regressão Polinomial, devemos fazer:

```
##### Regressão Polinomial #####
```

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree = 2)
previsores_treinamento_poly = poly.fit_transform(previsores_treinamento)
previsores_teste_poly = poly.transform(previsores_teste)
```

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
```

```
# Treinamento
regressor.fit(previsores_treinamento_poly, objetivo_treinamento)
```

```
# Teste
previsoes = regressor.predict(previsores_teste_poly)
```

Grau máximo do polinômio que pode ser usado.
Neste caso, o modelo que usaremos será o de uma parábola:

$$y = a_0 + b_1x_1 + b_2x_1^2$$

- **Regressão polinomial**

- Para aplicar a Regressão Polinomial, devemos

```
##### Regressão Polinomial #####
```

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree = 2)
previsores_treinamento_poly = poly.fit_transform(previsores_treinamento)
previsores_teste_poly = poly.transform(previsores_teste)
```

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
```

```
# Treinamento
```

```
regressor.fit(previsores_treinamento_poly, objetivo_treinamento)
```

```
# Teste
```

```
previsoes = regressor.predict(previsores_teste_poly)
```

previsores_teste_poly - Vetor do NumPy			
	0	1	2
0	1	28	784
1	1	58	3364
2	1	38	1444

previsores_teste - DataFrame	
Índice	idade
2	28
8	58
4	38

O que acontece, na verdade, é que aumentaremos o numero de variáveis previsoras: ao invés de usamos apenas o x , usaremos também o x^2 !

- **Regressão polinomial**

- Para aplicar a Re

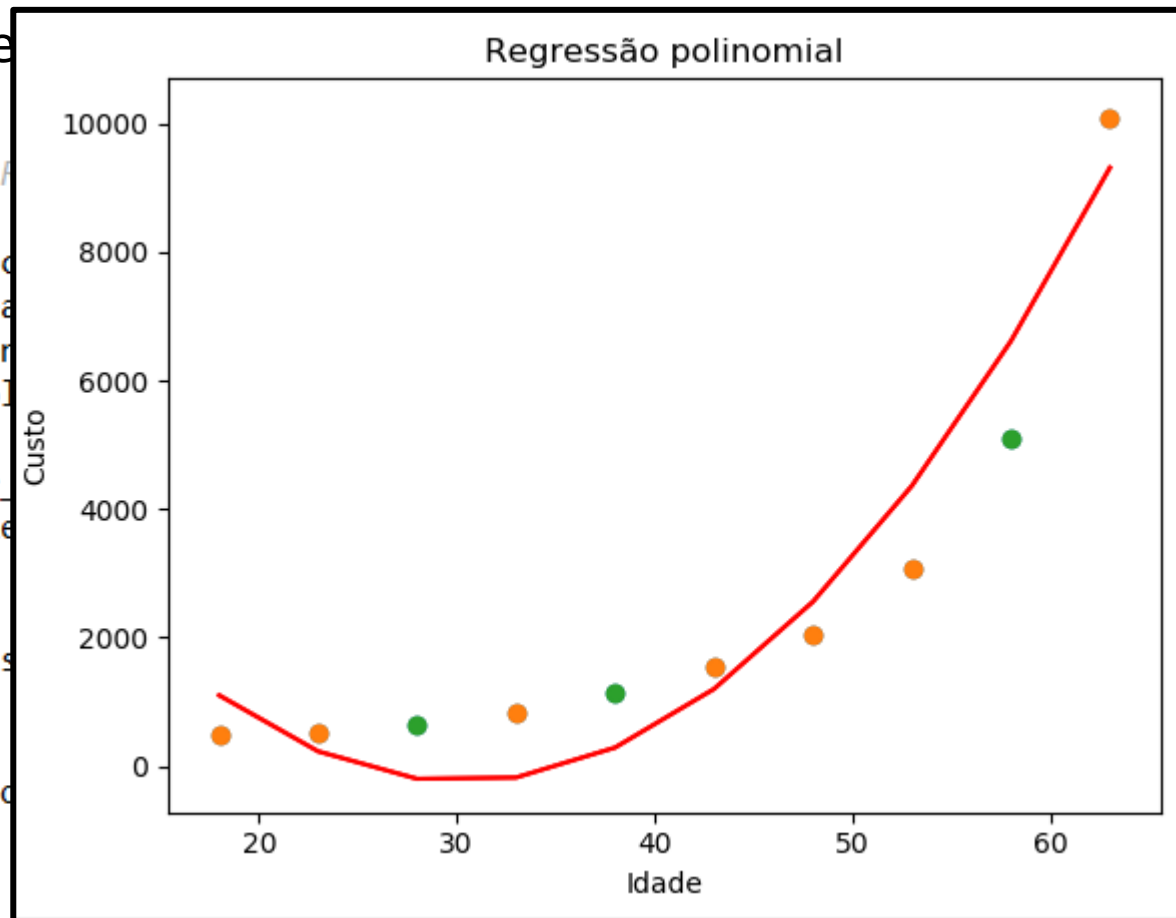
```
#####
```

```
from sklearn.preprocessing import PolynomialFeatures  
poly = PolynomialFeatures(degree=3)  
previsores_treina = poly.fit_transform(X_train)  
previsores_teste = poly.fit_transform(X_test)
```

```
from sklearn.linear_model import LinearRegression  
regressor = LinearRegression()
```

```
# Treinamento  
regressor.fit(previsores_treina, y_train)
```

```
# Teste  
previsoes = regressor.predict(previsores_teste)
```



O modelo fica mais próximos aos dados!

- **Regressão polinomial**

- Para aplicar a Regressão polinomial

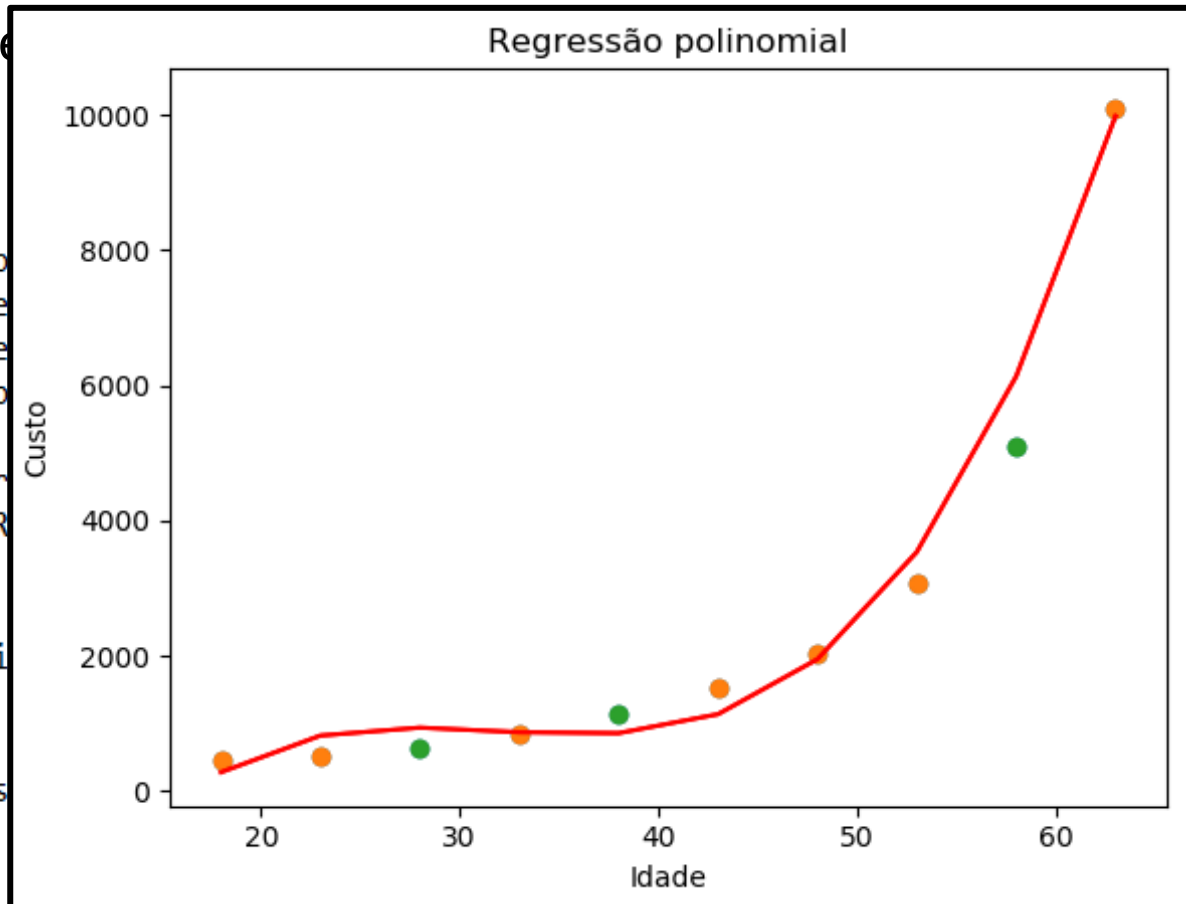
```
#####
```

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=3)
previsores_treiname = poly.fit(previsores_treiname, custo_treinamento)
previsores_teste_polynomial = poly.fit(previsores_teste, custo_teste)
```

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
```

```
# Treinamento
regressor.fit(previsores_treiname, custo_treinamento)
```

```
# Teste
previsoes = regressor.predict(previsores_teste_polynomial)
```

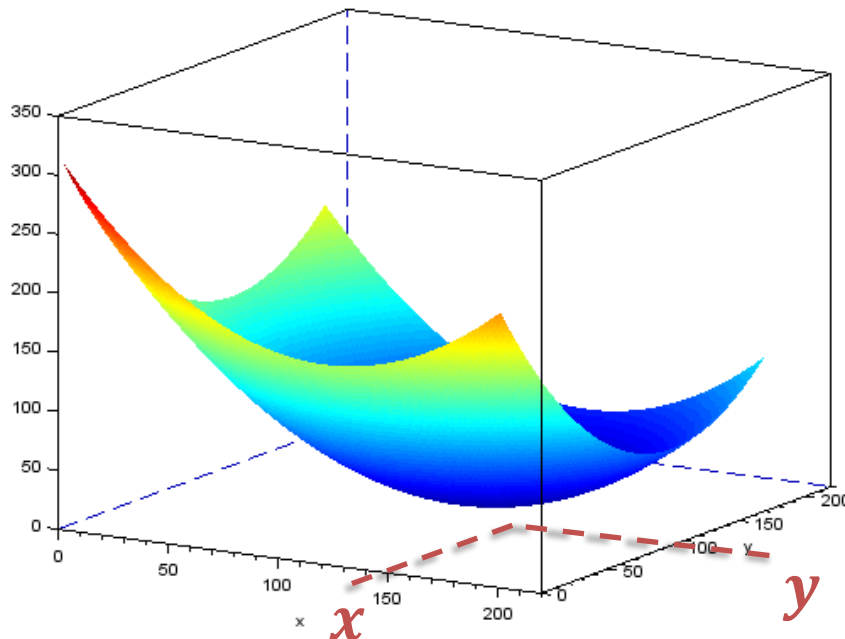


Aumentando o grau do polinômio para 3, a aderência fica ainda maior!

Porém, deve-se tomar cuidado com o overfitting!

Regressão Linear e Polinomial

- **Regressão polinomial múltipla**
 - Assim como fizemos com a Regressão Linear, podemos aplicar a Regressão Polinomial em problemas com **mais dimensões!**



$$f(x, y) = x^2 - 6x + y^2 - 4y + 13$$

Nesse caso, temos duas variáveis previsoras.

- **Regressão polinomial múltipla**

— Voltando ao problema dos preços de imóveis, o pré-processamento fica igual.

```
import pandas as pd

##### #Pré-processamento dos dados #####

#Leitura dos dados
base = pd.read_csv('house_prices.csv')
base.describe()

# =====
#                               Tratando valores inválidos
# =====
# Não há valores inconsistentes

# Procurando as colunas que possuem algum valor faltante
pd.isnull(base).any()

# =====
#                               Separando dados em previsores e objetivo
# =====

cols_previsores = ['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
                   'floors', 'waterfront', 'view', 'condition', 'grade', 'sqft_above',
                   'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode', 'lat', 'long']
# Não usei: date, sqft_living15 e sqft_lot15

cols_objetivo = ['price']
previsores = base[cols_previsores]
objetivo = base[cols_objetivo]

# =====
#                               Transformar as variáveis categóricas em valores numéricos
# =====
# Todas as variáveis são numéricas

# =====
#                               Separando em base de testes e treinamento
# =====

# usando 25% para teste
from sklearn.model_selection import train_test_split
previsores_treinamento, previsores_teste, objetivo_treinamento, objetivo_teste = train_test_split(previsores,
                                                                                                  objetivo,
                                                                                                  test_size=0.25,
                                                                                                  random_state=0)

# =====
#                               Padronização dos dados
# =====

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
previsores_treinamento = scaler.fit_transform(previsores_treinamento)
previsores_teste = scaler.transform(previsores_teste)
```

- **Regressão polinomial múltipla**

- Basta repetir o procedimento para aumentar o número de variáveis previsoras, usando um polinômio do grau desejado.

```
##### Regressão Polinomial #####
```

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree = 2)
previsores_treinamento_poly = poly.fit_transform(previsores_treinamento)
previsores_teste_poly = poly.transform(previsores_teste)
```

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
```

```
# Treinamento
regressor.fit(previsores_treinamento_poly, objetivo_treinamento)
```

```
score = regressor.score(previsores_treinamento_poly, objetivo_treinamento)
```

```
# Teste
previsoes = regressor.predict(previsores_teste_poly)
```

```
##### Avaliação dos resultados #####
```

```
score = regressor.score(previsores_teste_poly, objetivo_teste)
mae = metrics.mean_absolute_error(objetivo_teste, previsoes)
mse = metrics.mean_squared_error(objetivo_teste, previsoes)
rmse = np.sqrt(metrics.mean_squared_error(objetivo_teste, previsoes))
```

```
print('Score:', score)
print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

```
# Parâmetros estimados para o modelo
coef_0 = regressor.intercept_
coeficientes = regressor.coef_
```

- **Regressão**

- Basta repetir las predicciones, previsoras,

Vamos registrar os resultados em um arquivo único, para que possamos compará-lo com resultados futuros:

Score: 0.8083995465087881
Mean Absolute Error: 100200.43134715025
Mean Squared Error: 25452316878.014988
Root Mean Squared Error: 159537.8227193006

```
poly = PolynomialFeatures(degree = 2)
previsores_treinamento_poly = poly.fit_transform(previsores_treinamento)
previsores_teste_poly = poly.transform(previsores_teste)
```

[illegible]

Regressão Linear e Polinomial

FIM